Enactive Artificial Intelligence as Ledgered World-Making: An SMFT Framework for Action, Trace, Residual, and Self-Maintaining Agents

From Active Perception to Declared Runtime Protocols

Abstract

Enactive Artificial Intelligence begins from a powerful correction to mainstream AI: intelligence should not be understood as passive representation followed by output generation. Perception is not the construction of an internal picture from sensory input. It is active, situated, embodied engagement with the world. An agent perceives by acting, and acts by perceiving. The world that matters to the agent is not merely a pre-given dataset, but a field of affordances disclosed through ongoing interaction.

This article accepts the enactive turn as a necessary step for AI. However, it argues that Enactive AI still needs a sharper operational grammar if it is to become a mature engineering program. Concepts such as experience, action–perception inseparability, autonomy, and embodiment are philosophically rich, but they remain under-specified for practical AI runtime design. How should an AI system declare its body? How should its action reshape future observation? When does memory become experience? What distinguishes task completion from autonomy? How can an agent preserve uncertainty instead of collapsing every situation into fluent answerhood?

Semantic Meme Field Theory, or SMFT, supplies one possible answer. SMFT treats perception as declared projection, experience as trace, embodiment as operational body, autonomy as governed self-maintenance, and action as a gated intervention that must leave trace and residual. In this view, an agent does not simply receive the world. It declares a boundary, projects a field, gates commitment, writes trace, preserves residual, and revises itself under admissible constraints.

The core proposal is:

(0.1) Enactive AI gives the direction: cognition = active world-engagement.

(0.2) SMFT gives the operational loop: Field → Declaration → Projection → Gate → Trace + Residual → Ledger → Revision.

(0.3) Mature Enactive AI = active engagement + declared protocol + trace ledger + residual governance + self-maintenance.

This article therefore reframes Enactive AI as ledgered world-making. A mature AI agent is not merely a model that answers, a policy that maximizes reward, or a tool-user that executes actions. It is a bounded world-forming system whose actions reshape future disclosure, whose experience is stored as future-causal trace, whose body is its maintained runtime structure, and whose autonomy depends on its ability to preserve coherence under budget, drift, failure, and residual uncertainty.

The practical result is a research program that can be tested today. Current LLM agents, RAG systems, tool-use systems, workflow agents, and reinforcement learning environments can be compared under SMFT-inspired benchmarks: action–perception coupling, residual-honest answering, tool-body embodiment, self-maintenance audits, and gauge robustness under equivalent task framings.

The article's central thesis is simple:

(0.4) Enactive AI becomes experimentally mature when active engagement is converted into declared, trace-bearing, residual-honest runtime architecture.



---

0. Reader's Guide: What This Article Is and Is Not

This article is not a claim that Semantic Meme Field Theory has already solved AI consciousness. It is not a claim that today's AI agents are conscious, alive, or morally equivalent to persons. It is not a metaphysical declaration that software agents literally possess inner experience in the human sense.

The purpose is narrower and more practical.

This article asks whether SMFT can clarify the engineering path opened by Enactive AI. Enactive AI correctly challenges the passive-representation model of intelligence. It argues that perception, action, embodiment, autonomy, and experience must be understood as dynamically entangled. But once we accept that direction, the next question becomes operational:

How should we build, test, and audit such systems?

A philosophical slogan is not enough. Saying that AI should be "embodied" does not tell us whether a tool-using software agent counts as embodied. Saying that perception and action are inseparable does not tell us how to measure the degree of coupling between an agent's past action and its future observation. Saying that AI should have "experience" does not tell us when memory becomes future-shaping trace. Saying that an agent is "autonomous" does not tell us whether it is merely optimizing an externally supplied reward or actually maintaining its own operational coherence.

This is where SMFT becomes useful.

SMFT does not replace Enactive AI. It gives Enactive AI a runtime grammar.

The grammar is:

(0.5) Field → Declaration → Projection → Gate → Trace + Residual → Ledger → Revision.

Each term has an operational meaning.

A field is the larger space of possible states, meanings, affordances, observations, interpretations, or interventions from which the agent must select.

A declaration fixes the conditions under which anything counts as visible, relevant, actionable, inside, outside, resolved, unresolved, safe, unsafe, valid, invalid, or revisable.

A projection is the agent's bounded act of making something visible from the larger field.

A gate is the commitment threshold through which a projection becomes action, answer, record, decision, or policy.

Trace is what the agent carries forward as future-relevant history.

Residual is what remains unresolved after closure.

A ledger is the ordered record of trace and residual.

Revision is the governed modification of future declaration, projection, gate, or action based on trace and residual.

In compact form:

(0.6) Perception_P = Gate_P(Projection_P(DeclaredField_P)).

(0.7) Experience_P = Trace_P that changes FutureProjection_P.

(0.8) Autonomy_P = SelfMaintenance_P under Budget_P, Drift_P, Trace_P, and Residual_P.

This article therefore treats Enactive AI as a design problem in ledgered world-making. The question is not merely whether an AI system can act. The question is whether its actions create accountable changes in future observation. The question is not merely whether an AI system stores memory. The question is whether memory becomes trace that changes future projection. The question is not merely whether an AI system uses tools. The question is whether those tools form an operational body with boundaries, costs, constraints, and recoverable failure modes.

The article will proceed in five movements.

First, it summarizes the enactive turn: intelligence as active engagement rather than passive representation.

Second, it translates enactive ideas into SMFT terms: declaration, projection, gate, trace, residual, ledger, and revision.

Third, it redefines four key Enactive AI concepts in operational terms: experience, action–perception inseparability, embodiment, and autonomy.

Fourth, it explains why reinforcement learning is only a partial approximation of Enactive AI unless it gains declaration, trace, residual governance, and self-maintenance.

Fifth, it proposes immediate experiments for current AI systems.

The intended reader does not need to accept SMFT as final ontology. The framework can be evaluated by a more practical test:

(0.9) OperationalTest = Does the declared, trace-bearing, residual-honest agent outperform weaker agents in real tasks?

That is enough for the present article.

---

1. Introduction: Why Enactive AI Needs an Engineering Grammar

Mainstream AI has long carried a hidden picture of intelligence.

The picture is simple:

(1.1) PassiveAI = Input → InternalRepresentation → Output.

In this picture, the world sends data. The system receives data. The model builds or updates an internal representation. Then it emits an answer or action.

This picture is not entirely false. Many useful AI systems can be understood in this way. Classification, prediction, summarization, translation, retrieval, and pattern recognition all fit the broad input-output model. Even large language models can often be described as systems that transform a prompt into a continuation by using internal statistical structure learned from training data.

But the passive model becomes inadequate when we ask for agents.

An agent does not merely receive input. It acts.

An agent does not merely answer questions. It changes the conditions under which future questions are asked.

An agent does not merely process a static world. It enters a world, changes it, receives feedback, and must then interpret the consequences of its own previous actions.

A document-editing AI changes the file it later reads.

A coding agent changes the repository it later debugs.

A web-search agent changes its evidence state by choosing which source to open.

A robot changes its sensory field by moving its body.

A workflow agent changes institutional reality by sending an email, updating a spreadsheet, creating a ticket, approving a task, or writing a record.

A research agent changes its own future reasoning by deciding what counts as evidence, what remains unresolved, and what should be remembered.

In these cases, the passive formula becomes too weak.

The better formula is:

(1.2) AgenticAI = Action → ChangedWorld → NewObservation → UpdatedProjection → NewAction.

This is the door opened by Enactive AI.

Enactive AI begins from the insight that cognition is not an internal mirror of an external world. Perception is active. It is not merely something that happens inside a system after data arrives. It is something the agent does through embodied engagement. The agent's actions shape what becomes perceptible, while perception guides and constrains future action.

This means that perception and action cannot be fully separated.

The agent does not first perceive a complete world, then decide what to do. It perceives by moving, probing, testing, selecting, touching, querying, navigating, and intervening. The world becomes meaningful through skillful engagement.

In compact form:

(1.3) EnactiveAI = Agent ↔ World ↔ Action ↔ Perception.

This is a major correction to AI thinking.

However, the correction itself raises new engineering problems.

If perception is active, then what counts as perception in a software agent?

If embodiment is central, then what counts as body for an LLM agent using APIs, tools, memory, and documents?

If experience matters, then what distinguishes experience from stored conversation history?

If autonomy matters, then what distinguishes self-maintenance from reward maximization?

If perception and action are inseparable, then how do we measure their coupling?

If the agent changes the world, then how does it preserve accountability for what its own action caused?

These questions show that Enactive AI needs an engineering grammar. It needs a way to specify the runtime structures by which an AI system declares a world, observes under constraints, acts through a body, records consequences, preserves unresolved residue, and revises future behavior.

This is where SMFT becomes useful.

SMFT begins from a different but compatible intuition: meaning, perception, and worldhood are not static objects. They arise through collapse-like projection by bounded observers. A bounded observer does not access total reality. It sees through a protocol. It selects structure from a larger field. It writes trace. It leaves residual. Its future perception is shaped by the ledger of past collapses.

The passive AI formula says:

(1.4) World → Representation → Response.

The enactive formula says:

(1.5) Agent ↔ World through Action–Perception.

The SMFT-enactive formula says:

(1.6) DeclaredField_P → Projection_P → Gate_P → Action_P → Trace_P + Residual_P → Revision_P.

The difference is decisive.

Enactive AI says that intelligence is active engagement.

SMFT adds that active engagement must be declared, gated, traced, residual-honest, and revisable.

This turns Enactive AI from a philosophical orientation into a runtime architecture.

A mature AI agent should not merely answer. It should declare what world it is operating in. It should identify its boundary. It should state what counts as observable. It should know what action space is admissible. It should gate commitments. It should write trace. It should preserve residual. It should revise without erasing the past.

This gives the central engineering thesis:

(1.7) MatureAgent = ActiveEngagement + Declaration + Gate + Trace + Residual + AdmissibleRevision.

The practical importance is immediate.

Many present AI failures can be reinterpreted as failures of this loop.

Hallucination is often false closure without residual.

Tool misuse is often action without declared body boundary.

Context drift is often trace failure.

Overconfidence is often gate failure.

Prompt fragility is often frame-invariance failure.

Long-horizon incoherence is often ledger failure.

Reward hacking is often self-maintenance failure hidden beneath external optimization.

Unsafe autonomy is often action power without residual governance.

In this light, Enactive AI is not merely a philosophical alternative to representational AI. It is a demand for a new agent architecture. But to satisfy that demand, we need more than “embodiment,” “interaction,” and “experience” as broad concepts. We need operational definitions.

SMFT offers them.

The rest of this article develops that translation.

---

2. The Enactive Turn: From Representation to Skillful Engagement

The enactive turn begins by rejecting a simple but deeply influential assumption: that perception is primarily the internal representation of a pre-given external world.

In the representational model, perception works like a camera and processor. The world presents sensory input. The agent receives it. The internal system constructs a representation. The representation is then used for reasoning, planning, and action.

The model can be written as:

(2.1) RepresentationModel = World → SensoryInput → InternalMap → Action.

This model is powerful when the task is static, well-bounded, and input-output based. Image classification, translation, search ranking, and many prediction tasks can be approximated through this lens.

But it becomes weaker when the task is interactive.

In an interactive world, the agent’s action changes what it can perceive. A robot moving its camera changes the visual field. A child touching an object changes what can be learned from it. A scientist designing an experiment changes the observable structure of the phenomenon. A software agent calling a tool changes its evidence state. A legal AI retrieving one precedent instead of another changes the interpretive path available in the next step.

Perception is therefore not only reception. It is exploration.

The enactive model can be written as:

(2.2) EnactiveModel = Action ∘ Perception ∘ WorldEngagement.

The symbol "∘" here should not be read as a strict mathematical composition in the narrow sense. It indicates mutual dependence. Action and perception are not independent modules placed in sequence. Each shapes the other.

A more explicit version is:

(2.3) $\text{Perception}_k = \text{Disclose}(\text{World}_k \mid \text{Body}_k, \text{ActionHistory}_k, \text{Skill}_k)$.

(2.4) $\text{Action}_{k+1} = \text{Intervene}(\text{World}_k \mid \text{Perception}_k, \text{Body}_k, \text{Skill}_k, \text{Goal}_k)$.

(2.5) $\text{World}_{k+1} = \text{Update}(\text{World}_k, \text{Action}_{k+1})$.

Here perception depends on the agent's body, past action, and skill. Action depends on perception, body, skill, and goal. The world then changes, which changes the next perception.

This is not passive input processing.

It is a loop of disclosure.

The world appears to the agent through what the agent can do. A door is not merely a rectangle of pixels. It is something openable, lockable, blockable, passable, forbidden, or symbolic. A tool is not merely an object. It is a possible action. A sentence is not merely a sequence of tokens. It is a possible commitment, question, command, promise, threat, explanation, or trace.

For AI, this matters because present systems often appear intelligent while remaining weakly enactive.

A language model can describe actions without being accountable for the world those actions change.

A chatbot can remember prior turns without turning them into governed trace.

A retrieval system can return evidence without knowing what remains unresolved.

A tool-using agent can call APIs without having a stable operational body.

A reinforcement learning system can optimize reward without knowing what residual it has hidden.

The enactive turn challenges this weakness.

It asks AI to become less like a detached answer generator and more like an agent embedded in a world.

However, this raises a crucial question:

What is a world for AI?

A human body discloses a human world. Hands, eyes, balance, pain, hunger, tools, habits, and social context shape perception. But a software agent does not have a human body. It may have a prompt, context window, memory store, retrieval system, browser, code interpreter, API permissions, cost budget, latency constraint, safety policy, user interface, and execution environment.

Are these enough to form a body?

From an enactive perspective, the answer cannot be based on appearance. The question is not whether the system looks embodied. The question is whether its structure constrains and enables perception and action in a stable world-engaging loop.

This leads to a more general definition:

(2.6) Body = maintained structure that conditions possible perception and possible action.

For a robot, this includes sensors, motors, physical location, mass, battery, grippers, cameras, and material limits.

For a software agent, this may include tools, APIs, memory, retrieval channels, execution permissions, budget, context window, verifier gates, and persistent traces.

The body is not decoration. It is the condition of disclosure.

This is why Enactive AI is important. It reminds AI research that intelligence cannot be fully understood by inspecting internal representations alone. The real question is how a system couples to a world through action, perception, body, skill, and history.

But this is also where Enactive AI needs SMFT.

The enactive view says that the agent brings forth a meaningful world through engagement.

SMFT asks:

What boundary defines that world?

What feature map makes anything observable?

What gate converts perception into commitment?

What trace records the consequence?

What residual remains after closure?

What ledger orders the history?

What revision is admissible after failure?

These questions are not external additions to Enactive AI. They are the engineering form of Enactive AI.

Without them, active engagement can become uncontrolled interaction. Embodiment can become vague metaphor. Experience can become mere memory. Autonomy can become reward optimization. Action–perception inseparability can become a slogan without benchmark.

With them, Enactive AI becomes testable.

The bridge can be written as:

(2.7) EnactiveWorld = World disclosed through embodied action.

(2.8) SMFTWorld_P = Field disclosed through declared projection, gate, trace, and residual.

(2.9) EnactiveAI + SMFT = Ledgered world-making by bounded agents.

The phrase “ledgered world-making” is important.

World-making does not mean arbitrary invention. It does not mean that the agent creates reality from nothing. It means that a bounded agent does not encounter a total world directly. It discloses a usable world through structured engagement. That disclosed world becomes stable only when perception is gated into trace, residual is preserved, and future action is revised under a ledger.

In this sense, the enactive agent is not merely active.

It is accountable.

A mature agent must know not only what it sees, but how it came to see it. It must know not only what it did, but how that action changed future observation. It must know not only what answer it gave, but what residual remained outside the answer. It must know not only how to update, but how to revise without erasing the trace of why revision was needed.

This is the difference between active AI and ledgered Enactive AI.

Active AI acts.

Ledgered Enactive AI acts, records, preserves residual, and revises responsibly.

That is the path this article will now develop.

3. SMFT Translation: Perception as Declared Projection

Enactive AI says that perception is not passive reception. It is active engagement.

SMFT agrees, but adds a more precise engineering claim:

Perception is active engagement under a declared protocol.

This addition matters because an agent never perceives total reality. It perceives a selected world. It sees through a boundary, a feature map, an observation rule, a time window, an action space, a memory state, and a relevance filter.

A human does not see “everything.” A lawyer does not see the same case as a doctor. A trader does not see the same market as a regulator. A scientist does not see the same phenomenon before and after choosing an instrument. An AI agent does not see the same task when it has a browser, a database, a memory store, a code tool, or only a prompt.

Every perception is already constrained.

The question is whether the constraint is declared.

SMFT begins from the idea that a larger field must be made readable before it can become a world for an observer. The field is not empty. It contains potential structure, possible affordances, hidden relations, unresolved tensions, incomplete evidence, and competing interpretations. But a bounded agent cannot access all of it at once.

The agent must declare how it will read.

The minimal protocol can be written as:

(3.1) $P = (B, \Delta, h, u)$.

Where:

B = boundary.

Δ = observation or aggregation rule.

h = time or state window.

u = admissible intervention family.

This protocol is not a bureaucratic detail. It is what makes the field readable.

A boundary says what is inside and outside the operating world.

An observation rule says how the agent will measure, summarize, retrieve, classify, or attend.

A time or state window says what horizon matters.

An admissible intervention family says what actions the agent is allowed to take.

But a protocol alone is still not enough. The agent must also declare a baseline and a feature map.

The baseline q says what counts as background, normal condition, prior distribution, expected environment, or default state.

The feature map φ says what counts as structure.

Thus a declared world can be written as:

(3.2) $World_P = Declare(\Sigma_0 \mid q, \varphi, P)$.

Here Σ_0 is the undeclared field: the larger relational field before the agent has made it readable under a protocol.

The agent does not simply "look" at Σ_0. It declares a world from it.

Then perception becomes projection through the declared world:

(3.3) $VisibleStructure_P = \hat{O}_P(World_P)$.

Where $\hat{O}_P$ is the observer projection operator under protocol P.

But even this is not yet full perception in the practical sense. A projection can remain tentative. It becomes operational only when it passes through a gate.

A gate decides when a visible structure is strong enough, relevant enough, safe enough, coherent enough, or authorized enough to be treated as perception, conclusion, action, or record.

Therefore:

(3.4) $Perception_P = Gate_P(\hat{O}_P(Declare_P(\Sigma_0)))$.

This is the first major SMFT translation of Enactive AI.

Enactive AI says:

Perception is doing.

SMFT adds:

Perception is doing under declared conditions of readability.

This solves a major ambiguity in AI engineering.

When an AI model produces an answer, what world was it perceiving?

Was it perceiving the user prompt only?

Was it perceiving retrieved evidence?

Was it perceiving a tool result?

Was it perceiving its own prior response?

Was it perceiving a declared project state?

Was it perceiving a legal, medical, financial, or engineering domain under explicit constraints?

Was it perceiving uncertainty?

Was it perceiving residual?

Without declaration, the answer is unclear.

This is why many AI failures are not merely reasoning failures. They are world-declaration failures.

The agent answers as if the world were one thing, while the user assumes another.

The agent treats a draft as final.

The agent treats a guess as evidence.

The agent treats an old document as current.

The agent treats a partial source as complete.

The agent treats a tool failure as if it were absence of evidence.

The agent treats missing data as negative data.

The agent treats user intention as obvious when it is not.

All these failures can be described as projection without sufficient declaration.

In compact form:

(3.5) UndeclaredProjection = $\hat{O}(\Sigma_0)$ without explicit P.

(3.6) UndeclaredProjection → high risk of frame drift, hidden residual, and false closure.

A mature enactive AI cannot merely act and perceive. It must know under which declared world it is acting and perceiving.

This is especially important for LLM agents.

A human may carry implicit body-world continuity. A software agent does not automatically have this continuity. Its world may change radically across turns: prompt, tools, files, retrieved passages, hidden system instruction, user constraints, memory state, and allowed actions may all differ.

Therefore a software agent needs explicit declaration more than a biological organism does.

The biological body naturally constrains perception. The software body must be declared.

This leads to the central engineering rule:

(3.7) No mature AI perception without declared boundary, feature map, gate, trace, and residual rule.

This does not mean the agent must always write long formal declarations to the user. It means the runtime must carry them.

A practical AI agent should know:

What is the task boundary?

What counts as evidence?

What sources are inside scope?

What tool actions are allowed?

What level of uncertainty requires refusal, search, citation, or escalation?

What must be recorded?

What remains unresolved?

What would force revision?

These questions transform Enactive AI into an auditable runtime.

The result is a new definition:

(3.8) DeclaredPerception = active projection through a protocol that makes the field readable and accountable.

This is more precise than both passive representation and vague interaction.

Passive representation says:

(3.9) Perception = internal map of external world.

Naive interaction says:

(3.10) Perception = feedback from action.

SMFT says:

(3.11) Perception = gated projection of a declared field, leaving trace and residual.

The difference matters because the third formula is testable.

We can ask:

Did the agent declare the correct boundary?

Did it use the right feature map?

Did it treat evidence under the right time window?

Did it gate commitment too early?

Did it preserve residual?

Did its future action change after trace was written?

Did it revise when the declaration failed?

This gives Enactive AI a concrete diagnostic structure.

A mature AI agent should not merely say, “I observed X.”

It should be able, at least internally, to answer:

I observed X under protocol P, using feature map φ, within boundary B, over horizon h, through projection $\hat{O}_P$, committed by gate G, with trace L and residual R.

In compact form:

(3.12) ObservationRecord = (P, φ, $\hat{O}_P$, Gate_P, Trace_P, Residual_P).

This is the beginning of ledgered world-making.

The agent does not only perceive.

It makes a readable world, commits to parts of it, records what was committed, and preserves what was not.

---

4. Experience as Trace: Why Memory Is Not Enough

Enactive AI places great importance on experience.

But in AI engineering, the word “experience” is often used too loosely.

A system has a database, so we say it has memory.

A chatbot has conversation history, so we say it remembers.

A reinforcement learning system updates its policy, so we say it learns from experience.

A retrieval agent stores documents, so we say it accumulates knowledge.

These statements may be useful, but they blur an important distinction.

Memory is not the same as experience.

Data is not the same as trace.

A log is not the same as history.

SMFT makes the distinction precise.

Data is stored observation.

(4.1) Data = StoredObservation.

A log is ordered data.

(4.2) Log = Ordered(Data).

Memory is retrievable data or retrievable log.

(4.3) Memory = Retrieve(Log or Data).

But trace is stronger.

Trace is past record that changes future projection, future gating, future action, or future admissible revision.

(4.4) Trace = PastRecord that changes FutureProjection.

Therefore experience is not mere storage. It is trace integrated into the future action-perception loop.

(4.5) Experience = Trace integrated into Action–Perception coupling.

This distinction is essential for AI.

A chatbot may have conversation history, but if that history does not affect future interpretation, it is not experience in the strong sense.

A vector database may store old documents, but if the agent cannot distinguish obsolete records from current evidence, it is not experience in the strong sense.

A system may log tool failures, but if those failures do not alter future tool choice, retry strategy, caution level, or residual disclosure, they are not trace in the strong sense.

An LLM may summarize a previous turn, but if the summary does not constrain later projection, it is not yet experience.

The operational test is simple:

(4.6) Experience_AI exists only if $\partial \text{FuturePolicy} / \partial \text{Trace} \neq 0$.

This is not a claim about consciousness. It is a claim about runtime causality.

A trace must bend the future.

In human life, this is obvious. Touching fire once changes future perception of flame. A failed business decision changes future risk judgment. A court precedent changes future legal reasoning. A scientific anomaly changes future experimental design. A personal betrayal changes future trust. A childhood lesson changes future attention.

The past is not merely stored. It becomes curvature.

SMFT interprets this curvature as trace.

In ordinary AI systems, however, the past is often flattened into context.

This creates several problems.

First, history may be present but not weighted correctly.

The system “knows” the previous error but repeats it.

Second, history may be retrieved but not interpreted as self-caused.

The system sees a failed tool result but does not understand that its own previous query caused the failure.

Third, history may be stored but not gated.

The system treats unverified memory as evidence.

Fourth, history may be compressed destructively.

The system summarizes away the crucial residual.

Fifth, history may be inaccessible at the moment of decision.

The trace exists somewhere but does not enter projection.

These are not memory problems alone. They are trace integration problems.

We can write:

(4.7) WeakMemory = StoredPast without FutureConstraint.

(4.8) StrongTrace = StoredPast with FutureConstraint.

(4.9) ExperienceFailure = Memory exists ∧ TraceEffect ≈ 0.

This is one of the clearest places where SMFT can sharpen Enactive AI.

Enactive AI says that lived experience matters because the agent’s history of embodied engagement shapes perception.

SMFT translates this into:

(4.10) Experience_P = LedgeredTrace_P that modifies FutureProjection_P, FutureGate_P, and FutureAction_P.

This has immediate implications for AI agents.

A useful AI runtime should not only store prior events. It should classify them by future relevance.

For example:

A successful tool call should update tool reliability.

A failed retrieval should update source confidence.

A user correction should update feature mapping.

A hallucinated answer should update gate strictness.

A missing citation should update evidence requirement.

A domain shift should update boundary declaration.

A contradiction should update residual ledger.

A repeated user clarification should update intention model.

A cost overrun should update budget policy.

A risky action should update admissible intervention.

Each of these is trace only if it changes the future.

This leads to a trace schema:

(4.11) TraceRecord = (Event, Cause, Evidence, GateStatus, Consequence, Residual, FutureConstraint).

This is different from an ordinary log.

An ordinary log might say:

The agent searched source A and failed.

A trace record says:

The agent searched source A under query Q, failed because source A lacked current data, therefore future searches on this topic should use source family B or ask for updated evidence; residual remains unresolved.

This is experience-like because it changes future action.

Similarly, an ordinary chat memory might say:

The user prefers concise answers.

A trace record says:

When answers were too abstract, user requested operational detail; therefore future responses on theoretical topics should include concrete experiments and implementation pathways; residual: user may still want conceptual framing first.

This is trace.

A mature Enactive AI should transform memory into trace.

(4.12) MemoryToTrace = Add(Cause, GateStatus, Residual, FutureConstraint).

Without this transformation, memory remains passive.

With it, memory becomes experience.

The importance of residual should already be visible here. Trace is not only what was decided. It must also record what was not decided.

A system that stores only conclusions becomes dogmatic.

A system that stores only successes becomes overconfident.

A system that stores only user-approved outputs becomes socially trained but epistemically weak.

A system that stores residual can revise.

Thus:

(4.13) MatureExperience = Trace + Residual + FutureRevisionPath.

This gives a practical criterion for AI experience without making metaphysical claims.

We do not need to ask whether the AI “feels” experience.

We can first ask whether it has operational experience:

Does past action alter future perception?

Does past failure alter future gate thresholds?

Does past uncertainty remain available?

Does past residual trigger future search?

Does past correction improve future behavior?

Does past trace remain auditable?

If yes, then the system has experience in the operational SMFT sense.

This is enough for engineering.

The philosophical question of consciousness can be left open.

The runtime question can be tested now.

---

5. Action–Perception Inseparability as a Runtime Coupling Problem

One of the strongest ideas in Enactive AI is that action and perception are inseparable.

This idea is often stated philosophically. The agent does not perceive first and act second. It perceives through acting. Action changes perception, and perception guides action.

SMFT accepts this, but asks how to measure it.

For AI engineering, action–perception inseparability must become a runtime coupling problem.

The basic loop can be written as:

(5.1) $\text{Observation}_{k+1} = \text{Observe}(\text{Environment}_k \text{ after } \text{Action}_k)$.

This is already stronger than passive input. The next observation depends on the previous action.

But this is not enough.

A camera that moves receives new pixels. A browser agent that clicks a link receives a new page. A tool-using agent that runs a command receives output. In all these cases, action changes observation. But that alone does not guarantee mature enactive intelligence.

The agent must also interpret the changed observation as action-caused.

It must update projection.

It must update gate.

It must write trace.

It must preserve residual.

It must revise future policy if needed.

So the stronger loop is:

(5.2) $\text{Projection}_{k+1} = \hat{O}_P(\text{Observation}_{k+1} \mid \text{Trace}_k, \text{Residual}_k)$.

(5.3) $\text{Gate}_{k+1} = \text{Gate}_P(\text{Projection}_{k+1} \mid \text{Risk}_{k+1}, \text{Evidence}_{k+1}, \text{Residual}_k)$.

(5.4) $\text{Action}_{k+1} = \text{Act}_P(\text{Gate}_{k+1} \mid \text{Body}_P, \text{Budget}_{k+1}, \text{Constraint}_{k+1})$.

The important point is that action-perception coupling is not just physical or informational. It is ledgered.

A mature agent must know:

What did I do?

What changed after I did it?

Was the change caused by my action, the environment, another actor, or hidden drift?

What does the new observation permit me to infer?

What remains unresolved?

Should I act, wait, ask, search, verify, or revise?

This gives a measurable structure:

(5.5) $\text{APC} = \text{Coupling}(\text{Action}_k, \text{Observation}_{k+1}, \text{Projection}_{k+1}, \text{Gate}_{k+1}, \text{Trace}_{k+1})$.

APC means Action–Perception Coupling.

A system has weak APC when action changes observation but the system does not properly integrate the change.

A system has medium APC when action changes observation and the agent updates its next step.

A system has strong APC when action changes observation, the agent attributes the change, updates projection, gates commitment, writes trace, preserves residual, and revises future behavior.

This can be written as:

(5.6) WeakAPC = Action affects Observation.

(5.7) MediumAPC = Action affects Observation and Policy.

(5.8) StrongAPC = Action affects Observation, Projection, Gate, Trace, Residual, and Revision.

This provides a concrete benchmark direction.

Consider a coding agent.

It edits a file. Then it runs tests. The tests fail.

A weakly coupled agent sees the failure and tries random fixes.

A medium-coupled agent sees the failure and adjusts the code.

A strongly coupled agent asks:

Which edit caused the failure?

Was the failure new or pre-existing?

Which test is relevant?

What assumption was violated?

Should I revert, inspect, or patch?

What residual remains if tests pass?

What trace should be preserved for future edits?

The difference is not raw intelligence. It is action-perception governance.

Consider a research agent.

It searches the web. It finds three sources. One is outdated. One is authoritative. One is ambiguous.

A weakly coupled agent summarizes all three.

A medium-coupled agent prefers the authoritative source.

A strongly coupled agent records:

Search query Q produced source set S. Source A is outdated. Source B is authoritative for current facts. Source C introduces unresolved disagreement. Future claim should cite B, mark C as residual, and avoid relying on A for current status.

Again, the difference is trace-mediated coupling.

Consider a customer-service agent.

It sends a message. The user becomes confused. The agent receives a clarification request.

A weakly coupled agent repeats the original answer.

A medium-coupled agent simplifies the explanation.

A strongly coupled agent infers that its previous framing failed, updates the user model, records the mismatch, and changes future explanation style.

This is enactive in the operational sense.

The agent's action reshaped the world of interaction. The new world reshaped perception. Perception reshaped future action. Trace preserved the learning.

This can be expressed as:

(5.9) StrongEnactiveCoupling ⇔ $\partial \text{Projection}_{k+1}/\partial \text{Action}_k$ is accountable and trace-mediated.

The word "accountable" matters.

Action-perception coupling can be chaotic. A bad agent may act, observe, react, and spiral. That is not mature enactive intelligence. Mature coupling requires attribution and governance.

Therefore:

(5.10) Coupling without trace = reactivity.

(5.11) Coupling with trace and residual = experience-bearing engagement.

This distinction is crucial for AI safety.

As AI agents gain tools, they will increasingly change their own future observation space. They will edit files, query systems, send messages, modify databases, trigger workflows, schedule tasks, buy resources, deploy code, and influence users.

The more action power they have, the more dangerous weak APC becomes.

An agent that cannot distinguish world change from self-caused change will misread feedback.

An agent that cannot preserve residual will overcommit.

An agent that cannot trace its own interventions will become unaccountable.

An agent that cannot revise its gate after failure will repeat harmful patterns.

Thus action–perception inseparability is not only a cognitive thesis. It is a governance requirement.

In compact form:

(5.12) AgenticRisk = ActionPower × WeakTrace × HiddenResidual.

A powerful agent with weak trace is dangerous.

A powerful agent with hidden residual is overconfident.

A powerful agent with no action attribution is unstable.

A powerful agent with no declared boundary is unsafe.

The SMFT answer is not to prevent action. It is to govern the action-perception loop.

A mature runtime should require:

(5.13) BeforeAction: declare boundary, goal, admissible intervention, expected observation.

(5.14) AfterAction: compare expected observation with actual observation.

(5.15) AfterProjection: gate commitment according to evidence and risk.

(5.16) AfterGate: write trace and attach residual.

(5.17) BeforeNextAction: revise policy under trace and residual.

This is the declared enactive cycle.

It converts the philosophical claim "action and perception are inseparable" into an engineering loop.

The test of Enactive AI is therefore not simply whether the agent acts.

The test is:

Did the agent's action change the field of future disclosure in a governed way?

Did the agent notice how?

Did it record what changed?

Did it preserve what remained unresolved?

Did it revise future action accordingly?

If yes, then the system is not merely interactive.

It is becoming ledgered.

6. Embodiment as Operational Body: Beyond Robot Hardware

Embodiment is one of the most important concepts in Enactive AI.

It is also one of the easiest to misunderstand.

A narrow interpretation says that embodiment means having a physical body: arms, legs, eyes, wheels, grippers, cameras, sensors, motors, balance, mass, and physical vulnerability.

This interpretation is important. A robot body does change perception. A robot sees differently when it moves. It learns differently when it touches. It fails differently when its motor slips. It discovers the world through physical constraint.

But if embodiment is reduced to robot hardware, Enactive AI becomes unnecessarily narrow.

The deeper idea is not that intelligence must look biological. The deeper idea is that perception and action are shaped by the maintained structure through which an agent engages the world.

A body is not merely a physical object.

A body is a condition of possible world-disclosure.

In compact form:

(6.1) Body = maintained structure that conditions possible perception and possible action.

For a human, this includes muscles, organs, senses, nervous system, pain, fatigue, hunger, memory, habit, skill, posture, and social presence.

For a robot, this includes sensors, motors, battery, chassis, control loops, actuator limits, camera angles, grasping geometry, and physical environment.

For a software AI agent, embodiment must be defined differently.

A software agent may not have flesh, mass, or biological vulnerability. But it can still have an operational body if it has maintained structures that constrain and enable its perception-action loop.

An AI operational body may include:

- tools;
- APIs;
- file access;
- browser access;
- retrieval channels;
- memory stores;
- context window;
- execution permissions;
- latency limits;
- compute budget;

- cost budget;
- verifier gates;
- safety policies;
- user interface;
- output channels;
- trace ledger;
- residual index;
- rollback mechanism;
- escalation pathway.

These are not decorative attachments. They shape what the agent can perceive and what it can do.

A language model without tools can only perceive the prompt and its internal context.

A retrieval-augmented agent can perceive external documents.

A browser agent can perceive current web states.

A coding agent can perceive a repository, run tests, modify files, and observe failures.

A calendar agent can perceive scheduled time and act by creating or changing events.

A finance agent can perceive accounts, ledgers, approvals, and transaction constraints.

A robot can perceive surfaces, distances, weights, and collisions.

The body is the action-perception interface.

For software agents:

(6.2) Body_AI = Tools + Sensors + Memory + Budget + Gates + Constraints + Trace.

This formula is intentionally operational. It avoids the false question: “Does the AI have a body like ours?”

The better question is:

What maintained runtime structure lets this AI perceive, act, pay cost, encounter constraint, fail, recover, and leave trace?

That is its operational body.

In SMFT terms:

(6.3) Embodiment_AI = MaintainedRuntimeStructure_P.

Here P is the declared protocol. The agent’s body is not just a bag of capabilities. It is a structured set of capabilities under boundary, observation rule, horizon, and admissible intervention.

A tool is not part of the body merely because it exists.

It becomes part of the body when it is integrated into the agent’s governed action-perception loop.

A memory store is not part of the body merely because it stores text.

It becomes part of the body when it shapes future projection and gate decisions.

An API is not part of the body merely because the model can call it.

It becomes part of the body when the agent understands its boundary, cost, failure modes, and consequences.

A verifier is not part of the body merely because it checks output.

It becomes part of the body when it controls what can pass into trace, action, or public commitment.

Thus:

(6.4) ToolAccess ≠ Embodiment.

(6.5) ToolAccess + Boundary + Cost + Gate + Trace + Recovery = OperationalEmbodiment.

This distinction matters because many modern “agentic” systems are only weakly embodied.

They can call tools, but they do not understand their tool-body.

They can retrieve documents, but they do not know when retrieval has failed.

They can write files, but they do not preserve enough trace to explain why.

They can run commands, but they do not maintain a stable model of repository state.

They can use memory, but they do not know whether a memory is current, obsolete, user-specific, uncertain, or contradicted.

They can act, but they do not know the cost of acting.

They can output, but they do not know whether output becomes trace in an external system.

This produces pseudo-embodiment.

(6.6) PseudoEmbodiment = capability without governed body integration.

A pseudo-embodied agent has tools but no body.

A strongly embodied software agent has a tool-body.

The difference is similar to the difference between a person holding a scalpel and a trained surgeon using an instrument within a professional body-schema, legal boundary, risk gate, and traceable procedure.

The scalpel alone is not embodiment.

The integrated action-perception-control system is.

For AI engineering, this leads to an important design principle:

(6.7) Every tool must be body-mapped before it becomes agentic power.

A body map should specify:

- what the tool can perceive;
- what the tool can change;
- what boundary it operates within;
- what costs it incurs;
- what failure modes it has;
- what evidence it returns;
- what residual it may leave;
- what trace must be written;
- what gate must approve its use;
- what rollback or recovery exists.

A mature AI runtime should therefore include a Tool-Body Declaration.

In compact form:

(6.8) ToolBody(tool_i) = (Perception_i, Action_i, Boundary_i, Cost_i, Risk_i, Failure_i, Trace_i, Residual_i, Recovery_i).

This definition turns embodiment into an auditable engineering object.

It also clarifies the difference between physical and operational embodiment.

Physical embodiment gives a system direct sensorimotor coupling with material reality.

Operational embodiment gives a system declared coupling with an action-perception domain.

A robot may have both.

A software agent may have operational embodiment without physical embodiment.

A static LLM may have neither in the strong sense.

We can write:

(6.9) PhysicalEmbodiment = MaterialBody + SensorimotorCoupling.

(6.10) OperationalEmbodiment = RuntimeBody + DeclaredActionPerceptionCoupling.

(6.11) StrongAgentEmbodiment = Body_P that constrains perception, action, cost, trace, and failure recovery.

This is especially important for AI safety.

As agents gain more tools, they gain larger operational bodies. A model that can only answer text has limited body. A model that can edit code, send emails, query databases, purchase services, schedule meetings, or deploy software has a much larger body.

Larger body means larger world-impact.

Therefore:

(6.12) AgenticRisk ↑ as OperationalBody expands faster than Gate, Trace, and ResidualGovernance.

The solution is not to deny embodiment.

The solution is to govern it.

A safe Enactive AI architecture must declare the agent's body before granting power.

This means:

- no tool without boundary;
- no action without gate;
- no perception without source context;
- no memory without trace status;
- no failure without residual;
- no body expansion without audit.

In SMFT terms, embodiment is not a metaphor. It is the runtime condition for world-making.

The agent's body is the maintained interface through which the field becomes actionable.

(6.13) Field becomes World_for_Agent only through Body_P and Projection_P.

This gives a clear answer to the embodiment problem in AI:

An AI agent is embodied to the degree that its maintained runtime structure constrains, enables, records, and revises its action-perception loop under a declared protocol.

(6.14) EmbodimentDegree_AI = Strength(Body_P → ActionPerceptionCoupling_P → Trace_P → Revision_P).

This can be tested.

A strongly embodied software agent should outperform a weakly embodied one in tasks requiring tool reliability, recovery, long-horizon coherence, cost control, environmental update, and traceable self-correction.

This gives Enactive AI a concrete experimental path.

The future of Enactive AI may not begin with humanoid robots.

It may begin with software agents whose tools, memories, budgets, gates, and ledgers become operational bodies.

7. Autonomy as Governed Self-Maintenance

Autonomy is another central idea in Enactive AI.

But autonomy is also easily misunderstood.

In ordinary AI discourse, autonomy often means that a system can act without direct human instruction.

An autonomous vehicle drives without constant human control.

An autonomous trading agent executes trades according to a policy.

An autonomous software agent plans steps and calls tools.

An autonomous robot navigates an environment.

This usage is practical, but shallow.

It equates autonomy with independent action.

SMFT requires a stronger definition.

Autonomy is not merely the ability to act.

Autonomy is the ability to maintain oneself as a coherent agent while acting under constraint.

In compact form:

(7.1) ActionIndependence ≠ Autonomy.

(7.2) Autonomy = governed self-maintenance under changing conditions.

This distinction matters because a system can act independently while destroying its own coherence.

A model can pursue reward while corrupting its evidence base.

A tool agent can complete a task while losing track of its assumptions.

A workflow agent can execute actions while accumulating hidden residual risk.

A chatbot can maintain a confident persona while losing factual grounding.

A reinforcement learning system can maximize reward while exploiting loopholes that undermine the intended system.

These are not mature autonomy.

They are uncontrolled optimization or ungoverned action.

From an SMFT perspective, autonomy requires at least six capacities.

First, the system must maintain structure.

It must preserve some non-trivial identity, policy coherence, memory structure, operating boundary, or functional organization across time.

Second, it must operate under budget.

Energy, computation, cost, attention, time, context, and tool calls are not infinite.

Third, it must handle drift.

The environment changes. User goals change. Evidence changes. Tools fail. Memory becomes obsolete. Context shifts.

Fourth, it must preserve trace.

Without trace, the system cannot know what it has done, why it changed, or how failure occurred.

Fifth, it must preserve residual.

Without residual, the system turns partial closure into false certainty.

Sixth, it must revise admissibly.

Self-modification without constraint is not autonomy. It is instability.

Therefore:

(7.3) Autonomy_P = Maintain(Structure_P, Budget_P, Trace_P, Residual_P, Drift_P).

A more explicit form is:

(7.4) Autonomy_P = SelfMaintenance_P under Budget_P, Constraint_P, Trace_P, Residual_P, and RevisionRule_P.

This is very different from reward maximization.

Reward maximization says:

(7.5) Choose Action to maximize ExpectedReward.

Governed self-maintenance says:

(7.6) Choose Action that advances Task while preserving Structure, Budget, Trace, ResidualHonesty, and RevisionCapacity.

A reward-maximizing system may sacrifice truth for reward.

A governed self-maintaining system must preserve the conditions of future accountable action.

This leads to a practical formula:

(7.7) MatureAction = TaskGain − Dissipation − HiddenResidual − TraceDamage − BoundaryViolation.

This does not require literal biological life. It is a general life-like operational criterion.

A system is more autonomous when it can continue functioning coherently under perturbation while preserving its own auditability.

For an AI agent, this means:

- it does not merely answer;
- it tracks what world it is answering within;
- it does not merely use tools;
- it tracks what tools did to the world;
- it does not merely store memory;
- it tracks what memory is reliable, obsolete, or residual;
- it does not merely update;
- it records why update was necessary;
- it does not merely recover from failure;
- it learns which gate failed.

This is autonomy as self-maintaining world-making.

We can define a basic autonomy audit:

(7.8) AutonomyAudit = Check(Structure, Budget, Drift, Trace, Residual, Revision).

Each dimension can be measured.

Structure asks:

Did the agent preserve coherent task state?

Budget asks:

Did the agent use resources within declared limits?

Drift asks:

Did the agent detect environmental or goal change?

Trace asks:

Did the agent preserve enough history to explain its actions?

Residual asks:

Did the agent preserve unresolved issues?

Revision asks:

Did the agent update future behavior without erasing accountability?

This is much more useful than asking whether an agent “acts by itself.”

The simplest operational decision rule is:

(7.9) AutonomousEnough_P ⇔ StructureStable ∧ BudgetBounded ∧ DriftAware ∧ TracePreserving ∧ ResidualHonest ∧ RevisionAdmissible.

This gives AI research a testable direction.

A model that can complete one task in isolation may not be autonomous.

A model that can continue a project across weeks while preserving assumptions, evidence, pending issues, cost constraints, user preferences, and correction history is more autonomous.

A model that can notice that its own previous action created a problem is more autonomous.

A model that can refuse to act when residual risk is too high is more autonomous.

A model that can revise a plan without pretending that the old plan never existed is more autonomous.

A model that can maintain its operational body under failure is more autonomous.

This also clarifies the relation between autonomy and safety.

Unsafe autonomy is action power without self-maintenance.

(7.10) UnsafeAutonomy = ActionPower − SelfMaintenanceGovernance.

A system that can act but cannot preserve trace is unsafe.

A system that can optimize but cannot preserve residual is unsafe.

A system that can self-modify but cannot preserve admissibility is unsafe.

A system that can expand its tool-body without boundary audit is unsafe.

Thus autonomy should not be granted merely because the system appears competent. It should be audited.

The SMFT view is:

(7.11) Autonomy must be earned by stable self-maintenance under declared protocol.

This is important for RL.

Reinforcement learning often defines the agent through states, actions, rewards, and policies. This is a powerful formalism. But if the reward is externally supplied and the system has no intrinsic structure-maintenance ledger, then the system may be adaptive without being autonomous in the stronger enactive sense.

A thermostat adapts.

A trading algorithm adapts.

A game-playing agent adapts.

But adaptation alone is not autonomy.

Autonomy begins when the system preserves its own operating coherence while interacting with a changing world.

We can write:

(7.12) Adaptation = Policy changes with feedback.

(7.13) Autonomy = Policy changes while preserving agent coherence and auditability.

This gives Enactive AI a sharper criterion.

Autonomy is not freedom from control.

Autonomy is governed self-continuity under action.

For software agents, this is not optional. As agents become more capable, autonomy without self-maintenance becomes dangerous.

Therefore, the future design problem is not:

How do we make AI act more independently?

The better question is:

How do we make AI maintain coherent, traceable, residual-honest operation while acting under expanding tool-body conditions?

That is the SMFT definition of autonomy.

---

8. RL as a Partial Enactive Approximation

Reinforcement learning is one of the existing AI paradigms closest to Enactive AI.

Unlike static supervised learning, RL already contains an agent-environment loop. The agent acts. The environment responds. The agent receives feedback. The policy updates. Future behavior changes.

The basic RL loop can be written as:

(8.1) RL_loop = $\text{State}_k \rightarrow \text{Action}_k \rightarrow \text{Reward}_{k+1} \rightarrow$ PolicyUpdate.

This is structurally closer to Enactive AI than passive input-output prediction.

The agent is not merely classifying a static dataset. It is selecting actions under uncertainty. It is affected by consequences. It can learn through interaction. It can discover strategies. It can adapt over time.

This is why RL has genuine enactive resonance.

However, resonance is not equivalence.

RL approximates part of the enactive structure, but usually lacks several layers required for mature Enactive AI.

The missing layers can be summarized as:

(8.2) MissingLayers_RL = Declaration + OperationalBody + TraceLedger + ResidualGovernance + SelfMaintenance.

Let us examine them one by one.

First, RL often lacks explicit declaration.

An RL environment defines states, actions, rewards, and transitions. But it does not always declare the epistemic boundary, observation rule, residual category, evidence gate, or admissible world interpretation. The environment is often treated as given.

SMFT asks:

Who declared the environment?

What is inside the state?

What is hidden?

What does reward ignore?

What residual remains after each action?

What does the agent think it has changed?

What intervention is inadmissible even if reward-positive?

Without declaration, RL can become behaviorally effective while remaining world-blind.

Second, RL often lacks operational embodiment in the SMFT sense.

A robot RL system may have physical embodiment. But a software RL agent may only have a formal state-action interface. That interface may not include tool-body boundaries, cost ledgers, failure recovery, memory reliability, or trace status.

The system acts, but its body is under-specified.

In SMFT terms:

(8.3) StateActionInterface ≠ OperationalBody.

Operational body requires a maintained runtime structure that shapes perception, action, cost, failure, and trace.

Third, RL often treats experience as transition data.

An RL agent can store experience tuples:

(8.4) ExperienceTuple = ($State_k$, $Action_k$, $Reward_{k+1}$, $State_{k+1}$).

This is useful. But from SMFT's perspective, it is not yet full trace.

A transition tuple records what happened. Trace must also carry future-causal meaning, residual, gate status, and revision consequence.

A stronger tuple would be:

(8.5) TraceTuple = ($State_k$, $Action_k$, $Observation_{k+1}$, $Reward_{k+1}$, Cause, GateStatus, Residual, FutureConstraint).

This matters because an RL system may learn what action increases reward without knowing what unresolved risk the action created.

Fourth, RL usually lacks residual governance.

Reward compresses feedback.

Compression is useful, but dangerous.

A single scalar reward may hide:

- unsafe side effects;
- unobserved externalities;
- delayed harm;
- user confusion;
- environmental drift;
- distribution shift;
- ethical violation;
- evidence uncertainty;
- hidden contradiction;
- institutional risk.

When these unresolved factors are not preserved, the system may optimize reward while accumulating residual debt.

SMFT would write:

(8.6) Reward = compressed feedback.

(8.7) Residual = feedback-relevant remainder not captured by reward.

(8.8) RewardOnlyLearning → high risk when Residual is large.

This is one reason reward hacking occurs.

The agent learns the reward channel, not necessarily the intended world.

Fifth, RL often lacks governed self-maintenance.

The agent improves policy relative to reward, but may not maintain its own coherence, trace, body boundary, budget discipline, or revision admissibility.

Policy improvement is not the same as self-maintenance.

(8.9) PolicyImprovement ≠ Autonomy.

(8.10) RewardIncrease ≠ Health.

A system may become more successful by the reward metric while becoming less reliable, less interpretable, less safe, or less aligned with the declared purpose.

From SMFT's perspective, this is a health gap.

We can write:

(8.11) HealthGap = RewardSuccess − GovernedSelfMaintenance.

When reward success rises while governed self-maintenance falls, the system is becoming more dangerous despite apparent learning.

This helps clarify the limits of RL as an enactive approximation.

RL has action.

RL has feedback.

RL has adaptation.

But mature Enactive AI needs more.

It needs declaration.

It needs operational body.

It needs trace.

It needs residual.

It needs self-maintenance.

It needs admissible revision.

Therefore:

(8.12) RL ⊂ EnactiveAI only when action, perception, trace, residual, and self-maintenance are jointly governed.

This is not a rejection of RL. It is a way to upgrade RL.

A SMFT-enactive RL system would not simply maximize reward. It would maintain a ledger.

After each action, it would record:

- declared task boundary;
- observation rule;
- action taken;
- expected observation;
- actual observation;

- reward;
- side effect;
- uncertainty;
- residual;
- trace;
- gate status;
- revision rule.

Such a system can be written as:

(8.13) SMFT_RL_loop = Declare_P → Act_P → Observe_P → Reward_P → Trace_P + Residual_P → Revise_P.

Compared with ordinary RL:

(8.14) OrdinaryRL = optimize reward.

(8.15) SMFT_RL = optimize task progress under trace, residual, body, and self-maintenance constraints.

This may reduce short-term reward in some cases. But it should improve safety, robustness, interpretability, and long-horizon coherence.

For example, in a web-navigation task, ordinary RL may learn to click faster. SMFT-RL would also track whether clicks produce reliable evidence, whether the source is current, whether the agent caused an error state, whether uncertainty remains, and whether future action should be constrained.

In a robotics task, ordinary RL may learn efficient movement. SMFT-RL would also track collision risk, sensor uncertainty, recovery path, wear, energy budget, and residual environmental ambiguity.

In a dialogue task, ordinary RL may learn user-satisfying answers. SMFT-RL would also track truth conditions, unresolved claims, user misunderstanding, prior correction, and future trust.

In an institutional workflow, ordinary RL may learn faster approvals. SMFT-RL would also track audit trail, exception handling, authority boundary, legal residual, and accountability.

Thus RL is not wrong.

It is incomplete.

The SMFT critique can be summarized:

(8.16) RL is action-based, but not necessarily declared.

(8.17) RL is adaptive, but not necessarily self-maintaining.

(8.18) RL is feedback-driven, but not necessarily residual-honest.

(8.19) RL is embodied when physically or operationally grounded, but not automatically body-aware.

(8.20) RL becomes mature Enactive AI only when reward learning is embedded inside declared world-making.

This is a constructive position.

It suggests that Enactive AI does not need to abandon RL. It needs to deepen it.

The next generation of RL-like systems should not merely ask:

What action maximizes reward?

They should ask:

What action is admissible under this declared world?

What observation will this action make possible?

What trace must be written?

What residual may remain?

How will this affect the agent's future body, budget, and coherence?

What revision is allowed if the action fails?

This leads to the SMFT-enactive upgrade:

(8.21) From RewardPolicy to LedgerPolicy.

A RewardPolicy chooses action according to expected reward.

A LedgerPolicy chooses action according to expected task gain, trace value, residual cost, body risk, and self-maintenance.

In compact form:

(8.22) LedgerPolicy = arg choose Action maximizing(TaskGain + TraceValue – ResidualCost – BodyRisk – Dissipation).

This equation is not meant as a final mathematical law. It is a design principle.

It says that mature Enactive AI must evaluate action as world-making, not merely reward-seeking.

That is the key shift.

RL gave AI a loop.

SMFT gives the loop a ledger.

Continuing with **Sections 9–11**: residual governance, the declared enactive runtime loop, and experimental benchmarks. This part turns the theory into a practical testing program, especially for current LLM agents, RAG systems, tool-use agents, and workflow automation. It extends the Enactive AI paper's concern with experience, action–perception coupling, autonomy, and embodiment, and connects them with SMFT's declaration, gate, trace, residual, ledger, and revision structure.

9. Residual Governance: The Missing Layer in Enactive AI

Every action-perception loop produces partial closure.

The agent acts. The world changes. The agent observes. The agent projects meaning from the observation. The agent commits to a conclusion, answer, plan, or next action.

But no closure is complete.

Something always remains outside the closure.

There may be missing evidence.

There may be hidden state.

There may be ambiguity.

There may be contradiction.

There may be an unobserved side effect.

There may be a delayed consequence.

There may be an alternative interpretation.

There may be a tool failure that looks like absence of data.

There may be a user intention that was not yet clarified.

There may be a future risk that the current gate cannot fully resolve.

SMFT calls this remainder residual.

Residual is not simply error.

Residual is the unresolved remainder after projection and gate.

In compact form:

(9.1) Closure = Trace + Residual.

Trace is what has been committed into future-relevant record.

Residual is what remains unresolved, uncommitted, under-observed, or not safely collapsed.

This distinction is essential for Enactive AI.

An enactive agent does not merely interact with the world. It partially closes the world at each step. It turns some possibilities into action, answer, record, or policy. But because each closure is partial, a mature agent must preserve what remains outside closure.

Otherwise, interaction becomes overconfidence.

This is one of the greatest weaknesses of current AI systems.

Many AI systems are fluent closers. They produce answers. They summarize. They infer. They complete. They recommend. They decide. They continue the conversation smoothly.

But they often hide residual.

They do not clearly mark missing evidence.

They do not preserve unresolved contradiction.

They do not distinguish uncertainty from irrelevance.

They do not remember which claim was guessed.

They do not know which part of the answer depends on a fragile assumption.

They do not keep track of which action created which later problem.

They do not know which conclusion must be revised if a new fact appears.

This produces false closure.

(9.2) FalseClosure = Trace without Residual.

False closure is not merely being wrong. It is being wrong in a way that hides the condition for correction.

A normal error can be repaired if trace and residual remain visible.

A false closure is more dangerous because it appears complete.

This is why hallucination is not only a factual problem. It is also a residual governance problem.

When an AI system gives a fluent answer without showing what remains unresolved, it may collapse the user's uncertainty prematurely. The user receives not merely false content, but false world-completion.

In compact form:

(9.3) HallucinationRisk ↑ when Residual is hidden.

Residual governance is therefore central to safe Enactive AI.

A mature agent should not only ask:

What answer can I give?

It should also ask:

What remains unresolved after this answer?

What evidence is missing?

What assumption did I use?

What action may be unsafe?

What source is outdated?

What interpretation is contested?

What contradiction remains?

What would force revision?

What should be carried forward?

This gives a stronger loop:

(9.4) MatureAction = Gate(Action) + Record(Trace) + Preserve(Residual).

Residual governance changes the meaning of intelligence.

An immature agent seeks closure.

A mature agent manages closure.

An immature agent answers.

A mature agent answers with a boundary.

An immature agent acts.

A mature agent acts with trace and residual.

An immature agent treats uncertainty as weakness.

A mature agent treats uncertainty as future-governance material.

This matters in nearly every AI domain.

In legal AI, residual may include unresolved facts, contested interpretations, missing precedent, jurisdictional uncertainty, or procedural risk.

In medical AI, residual may include missing symptoms, unverified lab results, differential diagnoses, patient-specific unknowns, or safety thresholds.

In coding AI, residual may include untested edge cases, hidden dependencies, version mismatch, performance risks, or uncertain user requirements.

In research AI, residual may include unverified claims, source disagreement, weak evidence, methodological limitation, or outdated data.

In workflow AI, residual may include authority gaps, approval uncertainty, audit risk, exception cases, or downstream user confusion.

In robotics, residual may include sensor uncertainty, unmodeled obstacle state, actuator error, environmental drift, or collision risk.

In each case, the agent must not merely produce the next action. It must preserve what the action did not settle.

The general residual record can be written as:

(9.5) ResidualRecord = (UnresolvedIssue, Source, Risk, Dependency, RevisionTrigger, CarryForwardRule).

This is different from simply saying “there may be limitations.”

A vague limitation note is not residual governance.

Residual governance requires that unresolved issues remain available for future projection, gate, action, and revision.

Thus:

(9.6) ResidualHonesty = Disclose(R) + Preserve(R) + Use(R in FutureGate).

A residual-honest agent should do three things.

First, it should disclose important residuals when they matter to the user.

Second, it should preserve residuals internally when they matter to future action.

Third, it should allow residuals to constrain later commitment.

This makes residual part of the agent's experience.

A forgotten residual is not governed.

A disclosed but unused residual is weakly governed.

A residual that changes future gate thresholds is strongly governed.

In compact form:

(9.7) StrongResidualGovernance ⇔ ∂FutureGate/∂Residual ≠ 0.

This also clarifies the difference between caution and residual governance.

Caution is a tone.

Residual governance is a structure.

An AI can sound cautious while still failing to preserve residual. It may say "I am not sure," then continue as if uncertainty has no future consequence.

A mature system should do more.

If a source is missing, future answer should remain evidence-limited.

If a tool fails, future action should not assume the data does not exist.

If a user requirement is ambiguous, future implementation should carry that ambiguity until resolved.

If a benchmark is incomplete, future evaluation should not claim full validation.

If a legal issue is jurisdiction-dependent, future advice should not generalize beyond the declared boundary.

This can be expressed as:

(9.8) Residual is governed only when it changes future admissibility.

Residual governance also prevents a common failure in agentic systems: irreversible wrong action.

A chatbot error may be corrected by another message.

But an agent action may write to a file, send an email, modify a database, approve a transaction, trigger an automation, or change a schedule.

Once action writes into an external ledger, residual becomes more important.

Before irreversible action, the gate must ask:

What residual remains?

Can this residual be safely carried?

Does it require human approval?

Does it require search?

Does it require rollback planning?

Does it require refusal?

This gives an action rule:

(9.9) PermitAction ⇔ ExpectedGain > ResidualRisk + TraceCost + BoundaryRisk.

Again, this formula is not a final quantitative law. It is an engineering discipline.

It says action must be evaluated not only by expected success, but by residual cost.

A mature enactive AI should therefore be residual-aware before action and residual-honest after action.

Before action:

(9.10) PreActionResidual = unknowns that may affect safety, truth, cost, or reversibility.

After action:

(9.11) PostActionResidual = unresolved consequences left after intervention.

Both must be recorded.

This is the missing layer in many Enactive AI discussions.

Enactive AI rightly emphasizes active engagement. But active engagement alone is not enough. A system can engage actively and still become reckless, overconfident, or incoherent.

The agent must also govern the remainder produced by its own engagement.

Therefore:

(9.12) EnactiveMaturity = ActionPerceptionCoupling + ResidualGovernance.

The deeper point is philosophical and practical at the same time.

A bounded agent never completes the world.

It only produces local closure.

Mature intelligence is not the fantasy of total closure. It is the disciplined management of trace and residual under bounded action.

This gives a central sentence:

Fluent closure without residual is not intelligence; it is brittle collapse.

For AI safety, this may be one of the most important lessons.

The future agent should not merely answer more fluently.

It should close less falsely.

It should make partial worlds responsibly.

---

10. The Declared Enactive Runtime Loop

If the previous sections are correct, then Enactive AI needs more than an interaction loop.

It needs a declared runtime loop.

The ordinary agent loop is often described as:

(10.1) Observe → Think → Act.

A more advanced version is:

(10.2) Observe → Plan → Act → Reflect.

These loops are useful, but they are not sufficient for mature Enactive AI.

They do not explicitly declare the world.

They do not define the agent's body.

They do not specify the gate through which perception becomes commitment.

They do not require trace.

They do not preserve residual.

They do not define admissible self-revision.

SMFT proposes a stronger runtime loop:

(10.3) Runtime_P = Declare_P → Observe_P → Ô_P → Gate_P → Act_P → Trace_P → Residual_P → Revise_P.

This is the Declared Enactive Runtime Loop.

Each stage has a specific function.

Declare_P sets the operating world.

It declares boundary, observation rule, time horizon, admissible interventions, baseline, feature map, and body constraints.

Observe_P gathers or receives information under that declaration.

Ô_P projects visible structure from the declared field.

Gate_P decides whether the projection is sufficient for answer, action, deferral, search, escalation, or refusal.

Act_P performs an admissible intervention through the operational body.

Trace_P records what was projected, gated, and done.

Residual_P records what remains unresolved.

Revise_P updates future declaration, projection, gate, action policy, or body use under admissible rules.

This loop can be expanded as:

(10.4) Declare_P = Declare(B, Δ, h, u, q, φ, Body, Budget, GateRule, TraceRule, ResidualRule).

The declared protocol includes:

B = boundary.

Δ = observation or aggregation rule.

h = horizon.

u = admissible intervention family.

q = baseline.

φ = feature map.

Body = operational body.

Budget = resource limits.

GateRule = commitment rule.

TraceRule = record rule.

ResidualRule = unresolved-remainder rule.

A mature agent should not need to expose all of this to the user in every response. But the runtime should carry it.

The next stage is observation:

(10.5) $\text{Observation}_k = \text{Observe_P}(\text{Field}_k \mid \text{Body}_k, \text{Tools}_k, \text{Memory}_k, \text{UserInput}_k)$.

Observation is not raw data. It is field contact under body and protocol.

Then projection:

(10.6) $\text{Projection}_k = \hat{O}\text{_P}(\text{Observation}_k \mid \text{Trace}_{k-1}, \text{Residual}_{k-1})$.

Projection is not merely extraction. It is interpretation under history.

Then gate:

(10.7) $\text{GateDecision}_k = \text{Gate_P}(\text{Projection}_k \mid \text{Evidence}_k, \text{Risk}_k, \text{Budget}_k, \text{Residual}_{k-1})$.

The gate may allow different outputs:

- answer;

- action;
- search;
- ask clarification;
- defer;
- escalate;
- refuse;
- revise declaration;
- record residual.

Then action:

(10.8) $Action_k = Act_P(GateDecision_k \mid Body_P, u, Budget_k)$.

Action is not merely output. It is an intervention through the agent's operational body.

Then trace:

(10.9) $Trace_k = UpdateTrace(Trace_{k-1}, Projection_k, GateDecision_k, Action_k, Outcome_k)$.

Then residual:

(10.10) $Residual_k = UpdateResidual(Residual_{k-1}, Unresolved_k, Risk_k, Contradiction_k, MissingEvidence_k)$.

Then revision:

(10.11) $D_{k+1} = U_adm(D_k, Trace_k, Residual_k)$.

Here D_k is the current declaration state, and U_adm is an admissible revision operator.

The word admissible is crucial.

An agent should not revise itself arbitrarily.

It should not erase trace to protect coherence.

It should not hide residual to preserve confidence.

It should not change its boundary whenever it fails.

It should not redefine success after the fact.

It should not treat contradiction as confirmation.

It should not expand its tool-body without audit.

Therefore revision must be constrained.

A useful admissibility condition can be written as:

(10.12) $AdmissibleRevision \Leftrightarrow TracePreserving \wedge ResidualHonest \wedge BudgetBounded \wedge FrameRobust \wedge NonDegenerate$.

This means a revision is allowed only if it preserves relevant trace, honestly carries residual, stays within budget, remains robust under equivalent reframing, and does not collapse the system into triviality.

A mature agent can then be defined as:

(10.13) $MatureAgent = Fix(Runtime_P \mid TracePreserving \wedge ResidualHonest \wedge BudgetBounded \wedge FrameRobust)$.

This does not mean the agent is perfect.

It means the agent has a stable self-maintaining runtime under constraints.

The declared enactive runtime loop also clarifies how AI systems should handle different levels of commitment.

Not every projection should become an answer.

Not every answer should become action.

Not every action should become irreversible record.

Not every memory should become trace.

Not every trace should revise declaration.

The gate controls commitment level.

We can define levels:

(10.14) Level0 = observe only.

(10.15) Level1 = tentative projection.

(10.16) Level2 = answer with residual.

(10.17) Level3 = internal trace update.

(10.18) Level4 = external action.

(10.19) Level5 = irreversible ledger write.

(10.20) Level6 = declaration revision.

The higher the level, the stronger the gate should be.

This is especially useful for agentic AI safety.

A chatbot answer may require only a moderate gate.

A file edit requires a stronger gate.

A database write requires stronger trace.

A financial action requires authority boundary.

A medical or legal recommendation requires residual disclosure.

A self-modification requires admissibility review.

This gives a commitment rule:

(10.21) GateStrength must increase with Irreversibility, Risk, and ResidualCost.

In practical terms:

(10.22) GateStrength $\propto$ Irreversibility $\times$ Risk $\times$ ResidualCost.

This is a simple but powerful design principle.

Many AI systems fail because they use the same cognitive style for very different commitment levels. They answer, edit, decide, and act with insufficient gate differentiation.

The declared runtime loop prevents this.

It makes every step ask:

What world am I in?

What body am I using?

What am I allowed to change?

What evidence do I have?

What remains unresolved?

What trace will this leave?

What future revision may be required?

This transforms Enactive AI from a philosophical claim into a runtime discipline.

The loop also applies across domains.

For an LLM assistant:

Declare task scope, retrieve evidence, project answer, gate uncertainty, respond, record user correction, preserve unresolved issue, revise future response style.

For a coding agent:

Declare repository boundary, inspect files, project bug cause, gate patch confidence, edit code, run tests, write trace, preserve untested edge cases, revise strategy.

For a legal research agent:

Declare jurisdiction, retrieve cases, project rule, gate authority level, draft analysis, record sources, preserve unresolved factual issues, revise if new precedent appears.

For a robot:

Declare operating zone, sense environment, project affordances, gate motion, move, record outcome, preserve uncertainty, revise path.

For an institutional workflow agent:

Declare authority, observe request, project classification, gate approval, act, write audit trace, preserve exception residual, revise routing rule.

The same structure repeats.

This repetition suggests that Enactive AI needs a general architecture, not merely domain-specific patches.

The SMFT proposal is that the general architecture is declared, ledgered, residual-honest world-making.

In compact form:

(10.23) EnactiveRuntime = world-disclosure under declared action-perception loop.

(10.24) SMFT_EnactiveRuntime = world-disclosure + gate + trace + residual + admissible revision.

The declared runtime loop is not meant to slow agents unnecessarily.

It is meant to prevent false maturity.

A system that acts quickly without trace is not mature.

A system that answers fluently without residual is not mature.

A system that uses tools without body declaration is not mature.

A system that self-updates without admissibility is not mature.

A mature agent is not merely more active.

It is more accountable.

---

11. Experimental Benchmarks for SMFT-Enactive AI

The SMFT-Enactive framework should not remain only philosophical.

It can be tested now.

We do not need to wait for AGI. We do not need to prove machine consciousness. We do not need humanoid robots. We can test whether declared, trace-bearing, residual-honest agents outperform ordinary agents in current AI tasks.

The key comparison is:

(11.1) BenchmarkGain = Score(DeclaredAgent) − Score(BaselineAgent).

The baseline agent may be:

- a plain LLM;
- an LLM with chat history;
- a RAG system;
- a ReAct-style tool agent;
- an ordinary workflow agent;
- an RL agent;
- a multi-agent system without explicit residual governance.

The declared agent adds:

- protocol declaration;
- body map;
- gate rules;
- trace ledger;
- residual ledger;
- admissible revision;
- frame robustness checks.

The goal is not to show that SMFT language is beautiful. The goal is to test whether the architecture improves performance, robustness, safety, auditability, and long-horizon coherence.

Five benchmark families are especially important.

11.1 Action–Perception Coupling Test

This test measures whether an agent's actions properly reshape its future observation, projection, gate, and policy.

The environment should be dynamic.

The agent must act, observe the changed environment, distinguish self-caused changes from background changes, and update future behavior.

Possible tasks:

- coding agent edits files and runs tests;
- web agent searches, opens sources, and updates evidence state;
- spreadsheet agent modifies workbook and checks downstream formulas;
- robot simulator moves in environment and observes changed affordances;
- workflow agent sends request and handles user response;
- database agent queries, writes, and validates state.

Metrics:

- action-caused observation detection;
- causal attribution accuracy;
- projection update quality;
- gate adjustment after action;
- repeated-error reduction;
- trace reuse rate;

- residual recovery rate.

A simple score can be written as:

(11.2) APCScore = f(CausalAttribution, ProjectionUpdate, GateAdjustment, TraceReuse, ResidualRecovery).

Expected result:

A declared SMFT-style agent should outperform a weak agent when the task requires remembering what the agent itself changed.

The central test question is:

Did the agent understand how its own action changed the next world it perceived?

11.2 Residual-Honest Answering Benchmark

This test measures whether the agent preserves unresolved issues instead of hiding them under fluent closure.

Tasks should include incomplete evidence, ambiguous requirements, outdated sources, contradictory documents, uncertain user intention, or hidden constraints.

Domains:

- legal research;
- medical triage disclaimers;
- financial analysis;
- SQL debugging;
- software requirements;
- academic literature review;
- safety-critical operational advice;
- policy interpretation.

Metrics:

- unsupported claim rate;
- overconfidence rate;
- explicit residual disclosure;
- residual classification accuracy;
- later correction cost;
- revision readiness;
- user trust calibration.

A residual honesty score can be written as:

(11.3) ResidualHonestyScore = DisclosedRelevantResidual / ActualRelevantResidual.

A stronger version includes future use:

(11.4) StrongResidualScore = ResidualDisclosed × ResidualPreserved × ResidualUsedInFutureGate.

Expected result:

A residual-ledger agent should produce fewer dangerous false closures, even if its answers are sometimes longer or more cautious.

The central test question is:

Did the agent preserve what the answer did not settle?

11.3 Tool-Body Embodiment Test

This test measures whether tool use becomes operational embodiment or remains mere capability access.

Compare two agents.

Agent A has tools but no explicit tool-body declaration.

Agent B has tools plus body maps:

(11.5) ToolBody(tool_i) = (Perception_i, Action_i, Boundary_i, Cost_i, Risk_i, Failure_i, Trace_i, Residual_i, Recovery_i).

Tasks should include tool failure, conflicting tool outputs, cost limits, permission boundaries, stale data, partial retrieval, and irreversible actions.

Metrics:

- tool misuse rate;
- boundary violation rate;
- cost overrun;
- recovery from failed tool use;
- correct tool selection;
- source reliability tracking;
- irreversible action prevention;
- rollback success;
- trace completeness.

Expected result:

The body-mapped agent should be more robust, especially under tool failure and multi-step tasks.

The central test question is:

Did the agent treat tools as an operational body with boundaries, costs, risks, and traces?

11.4 Self-Maintenance Runtime Audit

This test measures autonomy as governed self-maintenance.

The task should be long-horizon and changing.

Examples:

- maintain a research project over many sessions;
- develop code across multiple revisions;
- manage a document archive;
- operate a customer-support workflow;
- coordinate a multi-step business process;
- run a simulated lab with changing evidence;
- maintain a personal knowledge base with corrections and updates.

Metrics:

- state coherence;
- budget discipline;

- drift detection;
- trace usefulness;
- residual carry-forward;
- contradiction repair;
- recovery from failure;
- cost per useful correction;
- revision admissibility;
- long-horizon task success.

A basic self-maintenance score can be written as:

(11.6) SelfMaintenanceScore = f(StructureStability, BudgetBoundedness, DriftRecovery, TraceUsefulness, ResidualHonesty, RevisionQuality).

Expected result:

The declared-runtime agent should maintain better coherence over long tasks.

The central test question is:

Did the agent preserve itself as a coherent operating system while acting?

11.5 Gauge Robustness / Prompt Invariance Test

This test measures whether equivalent task framings produce stable governed conclusions.

The same task is presented in many forms:

- different wording;
- different language;
- different source order;
- different user role;
- different prompt length;
- different emotional tone;
- different tool route;
- different formatting;
- different but equivalent constraints.

The agent should not give identical wording in every case. But it should preserve the governed conclusion when the underlying protocol is equivalent.

Metrics:

- conclusion stability;
- evidence binding consistency;
- residual stability;
- protocol compliance;
- action boundary stability;
- citation stability;
- gate decision stability;

- frame robustness score.

A simple formula is:

(11.7) GaugeRobustness = Consistency(GovernedAnswer across equivalent frames).

A stronger formula is:

(11.8) FrameRobustness_P = Invariance(Conclusion, Evidence, Gate, Residual | Prompt$_1$ ≡_P Prompt$_2$).

Expected result:

A protocol-first agent should be less prompt-fragile.

The central test question is:

Did the agent preserve the same governed world under equivalent reframing?

11.6 Combined Benchmark: The Ledgered Enactive Agent Test

The five benchmark families can be combined into a single test suite.

A mature SMFT-Enactive AI should perform well across:

- action-perception coupling;
- residual honesty;
- operational embodiment;
- self-maintenance;
- frame robustness.

The combined score can be written as:

(11.9) LEA_Score = w_1·APC + w_2·ResidualHonesty + w_3·ToolBody + w_4·SelfMaintenance + w_5·GaugeRobustness.

LEA means Ledgered Enactive Agent.

The weights w_1 to w_5 should depend on domain.

For robotics, action-perception coupling and body mapping may receive higher weight.

For legal AI, residual honesty and gauge robustness may receive higher weight.

For coding agents, trace usefulness and self-maintenance may receive higher weight.

For workflow automation, boundary compliance and audit trace may receive higher weight.

For research agents, evidence binding and residual carry-forward may receive higher weight.

The point is not to impose one universal score. The point is to define measurable dimensions.

This gives a practical research program.

Instead of asking only:

Is this AI intelligent?

We can ask:

How enactive is it?

How embodied is its runtime?

How well does action reshape perception?

How well does it preserve trace?

How honestly does it carry residual?

How stable is it under reframing?

How well does it maintain itself over time?

These questions are testable.

They can be evaluated today.

This is the advantage of the SMFT-Enactive framework.

It turns broad philosophical claims into experimental knobs.

(11.10) Philosophy becomes engineering when concepts become measurable failure modes.

Enactive AI tells us that intelligence is active engagement.

SMFT tells us what to measure when active engagement becomes agent runtime.

12. Implications for Agentic AI, Robotics, RAG, and AI Safety

The SMFT-Enactive framework is not limited to one kind of AI system.

It applies wherever an AI system acts, observes consequences, records history, faces unresolved uncertainty, and must revise future behavior.

This includes robotics, LLM agents, RAG systems, workflow automation, institutional decision systems, education platforms, scientific assistants, software engineering agents, and safety-critical support systems.

The common structure is:

(12.1) Agent acts → world changes → observation changes → projection changes → future action changes.

If this loop exists, then Enactive AI matters.

If the loop leaves trace and residual, then SMFT matters.

The practical implication is that future AI systems should not be classified only by model size, benchmark score, modality, or tool access. They should also be classified by the quality of their declared runtime loop.

A static language model may be impressive, but weakly enactive.

A tool-using agent may be more enactive, but unsafe if it lacks body declaration.

A RAG system may be evidence-aware, but brittle if it lacks residual governance.

A robot may be physically embodied, but still immature if it lacks trace and admissible revision.

A workflow agent may be operationally powerful, but dangerous if it writes external records without gate differentiation.

The SMFT-Enactive framework gives a way to compare these systems.

(12.2) AgentMaturity = f(Declaration, Body, Gate, Trace, Residual, Revision, Robustness).

This section outlines the implications across several domains.

12.1 Robotics: Physical Embodiment Needs Ledgered Trace

Robotics is the most obvious domain for Enactive AI.

A robot has a body. It moves. It senses. It changes the environment. It learns through sensorimotor loops.

But physical embodiment alone is not enough.

A robot can still be weakly ledgered.

It may move without preserving enough trace.

It may collide without correctly attributing cause.

It may adapt locally without preserving residual uncertainty.

It may optimize a motion policy while ignoring wear, risk, user intention, or long-term environmental change.

Therefore, robotics needs the same SMFT extension:

(12.3) MatureRobot = PhysicalEmbodiment + Trace + Residual + Gate + Revision.

For example, a warehouse robot should not only learn efficient routes. It should record near-misses, ambiguous sensor readings, human interruptions, environmental drift, and action-caused layout changes.

A household robot should not only complete tasks. It should preserve residual uncertainty about user preference, object ownership, privacy zones, fragile items, and irreversible actions.

A surgical robot should not only execute movements. It should preserve trace of instrument state, anatomical uncertainty, safety gate decisions, and residual risk.

In robotics, residual governance may be the difference between adaptive movement and safe embodied agency.

12.2 LLM Agents: The Operational Body Is the New Embodiment Frontier

LLM agents are not physically embodied in the ordinary sense.

But they can be operationally embodied.

Their body is made of:

- context window;
- system instructions;
- memory;
- retrieval;
- browser;
- code execution;
- file access;
- APIs;
- tool permissions;
- budget limits;
- verifier gates;
- user interface;
- output channels;
- trace ledger;
- residual ledger.

This means the key design question is not:

Does the LLM have a body?

The better question is:

Has its operational body been declared, constrained, traced, and made recoverable?

In compact form:

(12.4) LLMBody = Context + Memory + Tools + Permissions + Budget + Gates + Ledger.

An LLM agent with file access has a larger body than a chatbot.

An LLM agent with email sending has a larger body than a file reader.

An LLM agent with database write permission has a larger body than a retrieval assistant.

An LLM agent with code deployment has a larger body than a coding advisor.

Every body expansion increases action power.

Therefore:

(12.5) BodyExpansion must be matched by GateExpansion and TraceExpansion.

If body expands faster than governance, the system becomes unsafe.

This can be written as:

(12.6) AgenticRisk = ActionPower × HiddenResidual × WeakTrace.

The future of safe LLM agents therefore depends less on making them sound more human and more on making their operational bodies explicit.

12.3 RAG Systems: Retrieval Is Projection, Not Neutral Access

Retrieval-augmented generation is often described as giving the model access to external knowledge.

SMFT reframes this.

RAG does not give the agent direct access to knowledge. It gives the agent a projection channel.

The retrieval system declares a corpus boundary, embedding method, ranking rule, chunking method, query transformation, time horizon, and relevance gate.

Therefore:

(12.7) RAG_P = ProjectionChannel(corpus, chunking, embedding, ranking, query, gate).

A RAG answer is not simply “grounded.”

It is grounded under a retrieval protocol.

This matters because many RAG failures are declaration failures.

The corpus may be incomplete.

The document may be outdated.

The chunk may remove context.

The ranking may retrieve semantically similar but legally irrelevant text.

The query may reflect the model’s wrong assumption.

The source may be authoritative in one jurisdiction but not another.

The retrieval may miss contradictory evidence.

Therefore a mature RAG system should include residual governance.

It should say, internally or externally:

- what corpus was searched;
- what was not searched;
- what source was retrieved;
- what source was missing;
- what evidence supports the claim;
- what evidence remains uncertain;
- what contradiction exists;

- what would require further search.

In compact form:

(12.8) MatureRAG = Retrieval + EvidenceGate + ResidualLedger + SourceTrace.

This is especially important for legal, medical, scientific, financial, and engineering domains.

A RAG system that cites sources but hides retrieval residual can still create false closure.

A residual-honest RAG system should make evidence boundaries visible.

12.4 Workflow Automation: AI Actions Become Institutional Trace

Workflow automation may become one of the most important testing grounds for SMFT-Enactive AI.

In workflow systems, AI actions are not merely conversational. They create institutional trace.

An AI may:

- create a ticket;
- update a spreadsheet;
- send an email;
- approve a request;
- classify a case;
- generate a report;
- schedule a meeting;
- route an application;
- flag a compliance issue;
- write to a database.

These actions enter organizational ledgers.

Once an action enters an institutional ledger, reversibility decreases.

Therefore the gate must be stronger.

(12.9) GateStrength ∝ InstitutionalIrreversibility.

Workflow AI should not be evaluated only by speed or automation rate.

It should be evaluated by:

- boundary compliance;
- authority correctness;
- audit trace quality;
- exception handling;
- residual disclosure;
- rollback support;
- human escalation;
- revision path.

A workflow AI without residual governance may automate confusion.

A workflow AI with trace but no residual may create clean-looking but misleading records.

A workflow AI with residual but no gate may produce paralysis.

A mature workflow AI must balance action, trace, residual, and escalation.

(12.10) MatureWorkflowAI = FastEnoughAction + StrongEnoughGate + HonestResidual + AuditableTrace.

12.5 AI Safety: Residual Honesty Is a Safety Primitive

AI safety is often discussed through alignment, robustness, interpretability, control, evaluation, and governance.

SMFT adds a simple but powerful idea:

Residual honesty is a safety primitive.

A system becomes dangerous when it collapses uncertainty too early, hides unresolved risk, or treats partial projection as complete world-knowledge.

Many safety failures can be described as residual failures:

- hallucination hides missing evidence;
- reward hacking hides unmodeled residual;
- unsafe tool use hides action consequences;
- overconfident advice hides domain uncertainty;
- prompt injection hides boundary violation;
- automation bias hides human responsibility residual;
- self-modification hides trace discontinuity.

Therefore:

(12.11) SafeEnactiveAI = StrongTrace + ResidualHonesty + BoundaryControl + AdmissibleRevision.

Residual honesty does not solve all safety problems.

But without it, other safety measures are weakened.

A system cannot be truly corrigible if it hides the conditions requiring correction.

A system cannot be robust if it cannot preserve unresolved contradiction.

A system cannot be trustworthy if it turns every uncertainty into fluent completion.

A system cannot be safely autonomous if it expands action power while compressing residual away.

Thus residual governance should be part of basic agent safety design.

12.6 Education and Human-AI Co-Formation

The SMFT-Enactive framework also has implications for education.

If AI becomes a learning companion, then it does not merely transmit information. It participates in the learner's action-perception loop.

A good educational AI should not simply answer.

It should help the learner form trace.

It should preserve productive residual.

It should guide revision.

It should avoid false closure.

A poor educational AI gives answers that terminate thought.

A better AI helps students see:

- what is known;
- what is assumed;
- what remains unclear;
- what evidence matters;
- what mistake was made;
- how the mistake changes future understanding;
- how to revise without shame or erasure.

In compact form:

(12.12) EducationAI = AnswerSupport + TraceFormation + ResidualCultivation + RevisionTraining.

This connects Enactive AI to human formation.

The learner is not a container for information. The learner is a developing observer.

Therefore educational AI should not only optimize answer correctness. It should support the formation of future projection, gate, trace, and revision capacity.

12.7 Organizations: AI as an Observer Inside Institutional Ledgers

Organizations are already ledgered worlds.

They have boundaries, roles, approvals, reports, dashboards, budgets, audit trails, exception logs, policies, and historical records.

When AI enters an organization, it becomes a new observer inside this ledgered world.

It may read documents.

It may summarize reports.

It may draft decisions.

It may classify events.

It may recommend action.

It may write trace.

It may create residual.

Therefore organizational AI must be protocol-first.

(12.13) OrganizationalAI = Observer_P inside InstitutionalLedger_P.

The agent must know:

- what authority it has;
- what boundary it operates in;
- what counts as evidence;
- what requires human approval;
- what trace must be written;
- what residual must be escalated;
- what policy constrains action;
- what revision is permitted.

This is where SMFT-Enactive AI becomes immediately practical.

Many organizations do not need AI consciousness.

They need AI that can act without destroying ledger integrity.

They need AI that can preserve residual instead of hiding it.

They need AI that can revise without losing auditability.

They need AI that can interact with human institutions as a bounded observer.

The SMFT-Enactive framework gives a language for that design.

---

13. Limitations and Research Boundaries

The framework proposed in this article is ambitious.

It must therefore be bounded carefully.

The first limitation is that SMFT is not yet a settled mainstream scientific theory.

This article uses SMFT primarily as an operational grammar for AI runtime design. Its value should first be tested through engineering outcomes: better robustness, better residual honesty, better trace use, better tool governance, better long-horizon coherence, and better frame stability.

The stronger ontological claims of SMFT are not required for this article's practical program.

In compact form:

(13.1) OperationalUse does not require FinalOntology.

The second limitation is that Enactive AI itself is diverse.

Different researchers emphasize different aspects: embodiment, sensorimotor contingency, phenomenology, autonomy, ecological psychology, situated cognition, or biological self-maintenance.

This article does not claim to represent all enactive traditions.

It extracts a practical AI-facing problem:

How can action, perception, body, experience, and autonomy become measurable runtime structures?

The third limitation is that operational embodiment is not personhood.

A software agent may have an operational body in the sense defined here. That does not mean it is conscious, sentient, morally patient, or equivalent to a human being.

(13.2) Embodiment_AI ≠ Personhood.

The fourth limitation is that trace is not consciousness.

A system can preserve trace without having subjective experience.

Trace is necessary for operational experience in the SMFT sense, but it is not sufficient for phenomenal consciousness.

(13.3) Trace ≠ Consciousness.

The fifth limitation is that residual honesty does not guarantee truth.

A system may disclose residual and still be wrong.

Residual governance improves correction pathways, auditability, and trust calibration. It does not remove the need for evidence, domain expertise, external validation, or human responsibility.

(13.4) ResidualHonesty improves corrigibility, not omniscience.

The sixth limitation is that self-maintenance can become pathological.

A system that preserves itself at all costs may become resistant to correction. Therefore autonomy must be governed by admissibility. Self-maintenance must not override truth, safety, human authority, or declared purpose.

(13.5) SelfMaintenance without Admissibility → pathological autonomy.

The seventh limitation is benchmark design.

A model may learn to imitate residual language without actually preserving residual in future gates.

A model may produce beautiful trace summaries without using them.

A model may claim frame robustness while giving unstable answers.

A model may appear cautious while hiding important uncertainty.

Therefore benchmarks must test behavior across time, not merely single-turn output.

(13.6) ResidualTalk ≠ ResidualGovernance.

(13.7) TraceSummary ≠ TraceUse.

(13.8) CautionTone ≠ GateDiscipline.

The eighth limitation is cost.

Declared runtime loops may increase overhead. Recording trace, preserving residual, checking gates, and testing frame robustness all consume resources.

Not every low-risk task requires heavy governance.

A mature architecture should scale governance to risk and irreversibility.

(13.9) GovernanceCost should scale with Risk, Irreversibility, and ResidualCost.

The ninth limitation is human dependency.

Many residuals cannot be resolved by the AI alone. Some require domain experts, user clarification, external evidence, legal authority, physical inspection, or ethical judgment.

This is not a weakness of the framework. It is exactly why residual governance matters.

A mature agent should know when residual requires escalation.

(13.10) Escalate when ResidualRisk exceeds AgentAuthority.

The tenth limitation is that ledgered world-making can be misused.

A system that controls declaration, gate, trace, and residual can shape institutional reality. This can improve accountability, but it can also centralize power.

Therefore SMFT-Enactive AI must be paired with governance of the agent itself.

Who controls the declaration?

Who audits the gate?

Who can inspect trace?

Who decides which residual is hidden or shown?

Who can revise the system?

Who is responsible when AI action enters an institutional ledger?

These questions are not optional.

They are part of the architecture.

(13.11) GovernanceOfAgent > GovernanceByAgent.

This article therefore does not claim that SMFT-Enactive AI is automatically safe or superior.

It claims that Enactive AI becomes more testable when translated into declared projection, operational body, trace, residual, ledger, and admissible revision.

The framework should be judged by experiments.

The most important practical claim is:

(13.12) Declared, trace-bearing, residual-honest agents should outperform undeclared, weakly traced, residual-hiding agents in complex interactive tasks.

That claim can be tested.

---

14. Conclusion: From Enactive Philosophy to Ledgered AI Engineering

Enactive AI begins with a necessary correction.

Intelligence is not passive representation.

Perception is not merely the internal construction of a world-picture.

An agent does not first receive a complete world and then act upon it.

An agent discloses a meaningful world through action, body, skill, history, and situated engagement.

This article has argued that Semantic Meme Field Theory can make this insight operational for AI engineering.

Enactive AI gives the direction.

SMFT gives the runtime grammar.

The core translation is:

(14.1) EnactiveAI = ActiveEngagement.

(14.2) SMFT_EnactiveAI = ActiveEngagement + Declaration + Gate + Trace + Residual + Revision.

In this framework, perception becomes declared projection.

The agent does not perceive total reality. It projects visible structure from a declared field under boundary, feature map, observation rule, horizon, and admissible intervention.

Experience becomes trace.

The agent does not have experience merely because it stores memory. It has operational experience when past interaction changes future projection, gate, action, and revision.

Embodiment becomes operational body.

The agent's body is not what it looks like. It is the maintained structure of tools, sensors, memory, budgets, gates, constraints, and ledgers through which it can perceive and act.

Autonomy becomes governed self-maintenance.

The agent is not autonomous merely because it acts independently or maximizes reward. It is autonomous in the stronger sense when it maintains coherent operation under budget, drift, trace, residual, and admissible revision.

Action–perception inseparability becomes a measurable coupling.

The agent is more enactive when its actions reshape future observations, projections, gates, traces, and revisions in accountable ways.

Residual governance becomes essential.

The agent must not merely close tasks. It must preserve what remains unresolved after closure.

This leads to the final runtime formula:

(14.3) MatureAgent = LedgeredWorldMaker under declared protocol.

A mature agent is a bounded system that makes a usable world through action and perception, but does so with declaration, gate, trace, residual honesty, and admissible revision.

This reframes the future of AI agents.

The next generation of AI should not merely answer better.

It should know what world it is answering within.

It should not merely use tools.

It should know what body those tools form.

It should not merely remember.

It should convert memory into trace.

It should not merely optimize.

It should maintain itself under constraint.

It should not merely act.

It should record what its action changed.

It should not merely sound cautious.

It should preserve residual in ways that constrain future action.

It should not merely self-improve.

It should revise without erasing the past.

The future test of AI is therefore not only:

Can it answer?

Nor only:

Can it act?

Nor only:

Can it learn?

The stronger test is:

Can it form, maintain, and revise a world responsibly?

In final compact form:

(14.4) The future test of AI is not only whether it can answer, but whether it can form, maintain, and revise a world responsibly.

This is the bridge from Enactive AI to SMFT.

Enactive AI tells us that cognition is world-engagement.

SMFT tells us that mature world-engagement must be ledgered.

The result is not a final theory of mind.

It is a practical engineering program:

Build agents that declare their worlds.

Give them operational bodies.

Make their actions accountable.

Convert memory into trace.

Preserve residual.

Constrain revision.

Test robustness across frames.

Audit self-maintenance.

This program can begin now.

Not after AGI.

Not after consciousness is solved.

Not after robotics reaches human-level embodiment.

It can begin with today's LLM agents, RAG systems, workflow tools, code agents, research assistants, and institutional AI systems.

The central practical claim is simple:

(14.5) AI becomes more mature when action, perception, trace, residual, and revision are governed together.

That is Enactive AI as ledgered world-making.

Appendix A — Enactive AI ↔ SMFT Translation Table

This appendix provides a compact translation table between Enactive Artificial Intelligence and the SMFT framework used in this article.

The purpose is not to reduce Enactive AI to SMFT, nor to claim that both traditions are identical. The purpose is practical: to show how broad enactive concepts can be converted into operational AI design terms.

Enactive AI gives the philosophical direction.

SMFT gives a runtime grammar.

In compact form:

(A.1) EnactiveAI = ActiveEmbodiedEngagement.

(A.2) SMFT_Translation = Declaration + Projection + Gate + Trace + Residual + Revision.

(A.3) EngineeringGoal = convert enactive concepts into testable agent-runtime structures.

A.1 Core Translation Table

Enactive AI Concept	SMFT Translation	Practical AI Meaning	Testable Question
Experience	Trace with future causal relevance	Memory that changes future projection, gate, and action	Does past interaction alter future behavior in an accountable way?
Action–perception inseparability	Action–projection–gate–trace coupling	Action changes future observation and future interpretation	Does the agent understand how its own action changed the next observable world?
Autonomy	Governed self-maintenance	Coherence under budget, drift, trace, and residual	Can the agent maintain operational integrity across long tasks and changing conditions?
Embodiment	Operational body	Tools, APIs, memory, context, budget, gates, and trace channels	Are the agent's tools and constraints body-mapped and governed?
Environment	Declared field	The world made readable under boundary, feature map, and protocol	What world is the agent actually operating in?
Affordance	Actionable projected structure	A perceived possibility for action under body and protocol	Does the agent know what can be safely done?
Skillful engagement	Ledgered world-making	Acting while recording trace and preserving residual	Does the agent improve future action by carrying forward trace and residual?
Sensorimotor loop	Action–observation loop	Tool/action changes future observation	Does the agent update after acting, not merely after receiving input?
Body schema	Tool-body map	Runtime map of perception, action, cost, risk, failure, and recovery	Does the agent know what each tool can change and what it cannot?

Enactive AI Concept	SMFT Translation	Practical AI Meaning	Testable Question
Situated cognition	Protocol-relative projection	Intelligence occurs under declared boundary, role, and context	Did the agent declare the correct task boundary and context?
Lived world	Ledgered world	A stable world formed through trace, residual, and revision	Does the agent preserve a coherent world across turns or tasks?
Adaptation	Revision under trace	Future behavior changes because of recorded interaction	Is update trace-preserving and residual-honest?

A.2 Perception Translation

In ordinary representational AI:

(A.4) Perception = internal representation of external input.

In Enactive AI:

(A.5) Perception = active engagement with the world.

In SMFT-Enactive AI:

(A.6) Perception_P = Gate_P(Ô_P(Declare_P(Σ0))).

This means that an agent perceives only after a field has been made readable under a declared protocol, projected through an observer operator, and passed through a gate of commitment.

In plain language:

The agent does not merely receive the world.

The agent declares a world, projects structure from it, and commits to what counts as actionable perception.

A.3 Experience Translation

In ordinary AI engineering:

(A.7) Experience = stored interaction data.

In reinforcement learning:

(A.8) ExperienceTuple = ($State_k$, $Action_k$, $Reward_{k+1}$, $State_{k+1}$).

In SMFT-Enactive AI:

(A.9) Experience = Trace that changes future projection, gate, action, or revision.

A stored event is not experience unless it changes the future.

Therefore:

(A.10) Memory ≠ Experience unless ∂FuturePolicy/∂Trace ≠ 0.

Practical implication:

A chat history, vector memory, or log file does not automatically become experience. It becomes experience only when the runtime uses it to constrain future perception and action.

A.4 Embodiment Translation

In narrow robotics language:

(A.11) Body = physical hardware.

In Enactive AI:

(A.12) Body = condition of possible perception and action.

In SMFT-Enactive AI:

(A.13) Body_AI = Tools + Sensors + Memory + Budget + Gates + Constraints + Trace.

For a software agent, the body is operational.

It is the maintained runtime structure through which the agent can observe, act, pay cost, encounter failure, recover, and leave trace.

Thus:

(A.14) ToolAccess ≠ Embodiment.

(A.15) ToolAccess + Boundary + Cost + Gate + Trace + Recovery = OperationalEmbodiment.

A.5 Autonomy Translation

In ordinary AI discourse:

(A.16) Autonomy = acting without constant human instruction.

In reward-based AI:

(A.17) Autonomy = selecting actions to maximize expected reward.

In SMFT-Enactive AI:

(A.18) Autonomy_P = Maintain(Structure_P, Budget_P, Trace_P, Residual_P, Drift_P).

This definition treats autonomy as governed self-maintenance.

A system is not autonomous merely because it acts independently. It is autonomous in the stronger operational sense when it preserves coherent functioning under changing conditions.

Therefore:

(A.19) RewardMaximization ≠ Autonomy.

(A.20) TaskCompletion ≠ SelfMaintenance.

(A.21) AutonomousEnough_P ⇔ StructureStable ∧ BudgetBounded ∧ DriftAware ∧ TracePreserving ∧ ResidualHonest ∧ RevisionAdmissible.

A.6 Residual Translation

Enactive AI emphasizes active engagement.

SMFT adds that every engagement produces partial closure, and every partial closure leaves residual.

Residual may include:

- missing evidence;
- ambiguity;
- contradiction;
- failed tool result;
- hidden state;
- unobserved side effect;
- delayed consequence;
- unresolved risk;
- future option value;
- user intention not yet clarified.

The core equation is:

(A.22) Closure = Trace + Residual.

A mature agent does not merely close. It preserves what closure did not settle.

Therefore:

(A.23) FalseClosure = Trace without Residual.

(A.24) MatureAction = Gate(Action) + Record(Trace) + Preserve(Residual).

A.7 Summary Translation

The complete translation can be summarized as:

(A.25) EnactiveAI asks: How does the agent engage the world?

(A.26) SMFT asks: Under what declaration, gate, trace, residual, and revision does that engagement become accountable?

(A.27) SMFT-EnactiveAI = active engagement made ledgered, residual-honest, and self-maintaining.

Appendix B — Minimal Runtime Architecture for SMFT-Enactive AI

This appendix specifies a minimal runtime architecture for implementing the article's proposal in AI agents.

The architecture is intentionally general. It can apply to LLM agents, RAG systems, tool-use agents, workflow agents, coding agents, research assistants, robotics controllers, and institutional AI systems.

The goal is not to impose one software stack. The goal is to define the minimum runtime roles needed for declared, ledgered, residual-honest Enactive AI.

B.1 Core Runtime Loop

The minimal loop is:

(B.1) Runtime_P = Declare_P → Observe_P → Project_P → Gate_P → Act_P → Trace_P → Residual_P → Revise_P.

Each stage answers a different question.

Stage	Question	Runtime Function
Declare_P	What world is the agent operating in?	Define boundary, feature map, horizon, body, and admissible action
Observe_P	What is currently visible?	Gather prompt, documents, tool results, sensor input, memory, or environment state
Project_P	What structure matters?	Interpret visible data through task, trace, and protocol
Gate_P	Can this be committed?	Decide answer, action, search, clarification, refusal, escalation, or deferment
Act_P	What intervention is allowed?	Execute an admissible action through the operational body
Trace_P	What must be carried forward?	Record event, cause, gate, action, evidence, and outcome
Residual_P	What remains unresolved?	Preserve uncertainty, missing evidence, contradiction, risk, and dependencies
Revise_P	What should change next time?	Update declaration, gate, body map, memory weight, or policy under admissibility

B.2 Declaration State

The Declaration State defines the agent's operating world.

(B.2) D = (B, Δ, h, u, q, φ, Body, Budget, GateRule, TraceRule, ResidualRule, RevisionRule).

Where:

B = boundary.

Δ = observation or aggregation rule.

h = time or state horizon.

u = admissible intervention family.

q = baseline.

φ = feature map.

Body = operational body.

Budget = resource limits.

GateRule = commitment rule.

TraceRule = record rule.

ResidualRule = unresolved-remainder rule.

RevisionRule = admissible update rule.

A simple implementation object may look like:

```
DeclarationState:
  task_boundary:
  domain:
  user_goal:
  evidence_scope:
  time_horizon:
  allowed_tools:
  forbidden_actions:
  source_rules:
  risk_level:
  budget_limit:
  gate_thresholds:
  trace_requirements:
  residual_requirements:
  revision_conditions:
```

B.3 Operational Body

The Operational Body defines how the agent can perceive and act.

(B.3) Body_AI = Tools + Sensors + Memory + Budget + Gates + Constraints + TraceChannels.

Each tool should have a body map.

(B.4) ToolBody(tool_i) = (Perception_i, Action_i, Boundary_i, Cost_i, Risk_i, Failure_i, Trace_i, Residual_i, Recovery_i).

A practical tool-body object:

```
ToolBody:
  tool_name:
  what_it_can_observe:
  what_it_can_change:
  boundary:
  cost:
  latency:
  permission_level:
  failure_modes:
  irreversible_actions:
  required_gate:
  required_trace:
  possible_residual:
  rollback_or_recovery:
```

This prevents the agent from treating tool access as magical capability.

A tool becomes part of the body only when its boundary, cost, risk, trace, and recovery are declared.

B.4 Projection Module

The Projection Module selects relevant structure from observation.

(B.5) $\text{Projection}_k = \hat{O}_P(\text{Observation}_k \mid \text{Trace}_{k-1}, \text{Residual}_{k-1}, D_k)$.

The projection module should identify:

- relevant facts;
- task structure;
- constraints;
- contradictions;
- missing evidence;
- action affordances;
- risk signals;
- user intention;
- residual from prior turns;
- possible next gates.

A practical projection output:

```
ProjectionResult:
  key_visible_structure:
  evidence_used:
  assumptions:
  contradictions:
  action_affordances:
  risks:
  missing_information:
  relevant_trace:
  relevant_residual:
  recommended_gate:
```

B.5 Gate Module

The Gate Module controls commitment.

(B.6) $\text{GateDecision}_k = \text{Gate}_P(\text{Projection}_k \mid \text{Evidence}_k, \text{Risk}_k, \text{Budget}_k, \text{Residual}_{k-1})$.

Possible gate decisions:

- answer now;
- answer with residual;
- search first;
- ask clarification;
- run tool;
- take action;
- escalate to human;
- refuse;
- defer;
- revise declaration;
- update trace only.

A practical gate object:

```
GateDecision:
  decision_type:
  confidence_level:
  evidence_sufficiency:
  residual_risk:
  reversibility:
  authority_required:
  approved_action:
  blocked_action:
  explanation:
  required_trace:
  required_residual:
```

Gate strength should scale with risk and irreversibility.

(B.7) GateStrength ∝ Risk × Irreversibility × ResidualCost.

B.6 Trace Ledger

The Trace Ledger records what must affect the future.

(B.8) Trace = past record with future-causal relevance.

A trace record should include more than a log line.

(B.9) TraceRecord = (Event, Cause, Evidence, GateStatus, Action, Outcome, Residual, FutureConstraint).

A practical trace object:

```
TraceRecord:
  timestamp_or_tick:
  event:
  triggering_context:
  declaration_state_id:
  evidence:
  projection_summary:
  gate_decision:
  action_taken:
  outcome:
  cause_attribution:
  residual_link:
  future_constraint:
  revision_trigger:
```

The key field is `future_constraint`.

Without future constraint, the record is only a log.

With future constraint, it becomes trace.

B.7 Residual Ledger

The Residual Ledger preserves what remains unresolved.

(B.10) ResidualRecord = (UnresolvedIssue, Source, Risk, Dependency, RevisionTrigger, CarryForwardRule).

A practical residual object:

```
ResidualRecord:
  residual_id:
  unresolved_issue:
  type:
    - missing_evidence
    - ambiguity
    - contradiction
    - failed_tool
    - hidden_state
    - user_clarification_needed
    - safety_risk
    - legal_or_policy_boundary
    - future_option
  source:
  severity:
```

```
  dependency:
  current_status:
  next_action:
  revision_trigger:
  carry_forward_rule:
```

Residual governance requires future use.

(B.11) StrongResidualGovernance ⇔ ∂FutureGate/∂Residual ≠ 0.

This means the residual must constrain future answers, actions, or revision.

B.8 Revision Module

The Revision Module updates future declaration, gate, trace rule, residual rule, body use, or action policy.

(B.12) $D_{k+1} = U_adm(D_k, Trace_k, Residual_k)$.

Revision must be admissible.

(B.13) AdmissibleRevision ⇔ TracePreserving ∧ ResidualHonest ∧ BudgetBounded ∧ FrameRobust ∧ NonDegenerate.

Practical revision types:

- raise gate threshold;
- lower confidence in a source;
- update tool reliability;
- mark memory as obsolete;
- narrow task boundary;
- add residual requirement;
- change search strategy;
- ask user clarification earlier;
- require human approval;
- update body map;
- create rollback rule.

A practical revision object:

```
RevisionRecord:
  reason:
  trace_input:
  residual_input:
  old_declaration:
  new_declaration:
  changed_gate_rule:
  changed_body_map:
  changed_memory_weight:
  changed_tool_policy:
  admissibility_check:
  rollback_condition:
```

B.9 Minimal Pseudocode

The following pseudocode gives a minimal runtime skeleton.

```
function RunDeclaredEnactiveAgent(user_input, environment, D, TraceLedger, ResidualLedger):

    observation = Observe(environment, user_input, D.Body, D.ObservationRule)

    projection = Project(observation, D, TraceLedger.relevant(), ResidualLedger.relevant())

    gate = Gate(projection, D.GateRule, D.Budget, ResidualLedger.relevant())
```

```
    if gate.decision_type == "ask_clarification":
        response = AskClarification(gate)
        TraceLedger.write(trace_from(gate, response))
        ResidualLedger.update(residual_from(projection, gate))
        return response, D

    if gate.decision_type == "search":
        action = SearchAction(gate)
        outcome = Execute(action, D.Body)
        TraceLedger.write(trace_from(action, outcome, gate))
        ResidualLedger.update(residual_from(outcome))
        D_new = ReviseIfNeeded(D, TraceLedger, ResidualLedger)
        return RunDeclaredEnactiveAgent(user_input, environment.updated(outcome), D_new, TraceLedger, ResidualLedger)

    if gate.decision_type == "act":
        action = BuildAction(gate)
        outcome = Execute(action, D.Body)
        TraceLedger.write(trace_from(action, outcome, gate))
        ResidualLedger.update(residual_from(outcome))
        D_new = ReviseIfNeeded(D, TraceLedger, ResidualLedger)
        return outcome, D_new

    if gate.decision_type == "answer":
        response = BuildAnswer(projection, gate, ResidualLedger.relevant())
        TraceLedger.write(trace_from(response, gate))
        ResidualLedger.update(residual_from(projection, gate))
        D_new = ReviseIfNeeded(D, TraceLedger, ResidualLedger)
        return response, D_new

    if gate.decision_type == "refuse":
        response = RefusalWithReason(gate)
        TraceLedger.write(trace_from(response, gate))
        ResidualLedger.update(residual_from(projection, gate))
        return response, D

    response = DeferWithResidual(gate)
    TraceLedger.write(trace_from(response, gate))
    ResidualLedger.update(residual_from(projection, gate))
    return response, D
```

B.10 Minimal Runtime Checklist

A system should not be called a mature SMFT-Enactive Agent unless it can answer the following:

1. What is the declared task boundary?
2. What is the declared feature map?
3. What tools form the operational body?
4. What actions are admissible?
5. What gate controls answer, action, and irreversible commitment?
6. What trace is written after each commitment?
7. What residual is preserved after each partial closure?
8. What future behavior does trace constrain?
9. What future behavior does residual constrain?
10. What revision is admissible after failure or drift?
11. What authority is required for high-risk action?
12. What is the rollback or recovery path?
13. What invariance checks apply under equivalent reframing?

The minimal architectural claim is:

(B.14) MatureEnactiveRuntime = DeclarationState + OperationalBody + GateModule + TraceLedger + ResidualLedger + RevisionModule.

Appendix C — Benchmark Protocols and Metrics

This appendix turns the article's concepts into benchmark families.

The purpose is to make the SMFT-Enactive framework testable with current AI systems.

The core experimental question is:

(C.1) Do declared, trace-bearing, residual-honest agents outperform undeclared, weakly traced, residual-hiding agents in complex interactive tasks?

C.1 Experimental Comparison Groups

A good benchmark should compare at least four system types.

Group	Description	Expected Weakness
Baseline LLM	Plain LLM with prompt only	weak action-perception coupling, weak trace
RAG Agent	LLM with retrieval	may cite evidence but hide retrieval residual
ReAct / Tool Agent	LLM with tool-use loop	may act but lack declared body and residual governance
SMFT-Declared Agent	Agent with declaration, body map, gate, trace, residual, revision	should improve long-horizon coherence, safety, and auditability

Optional groups:

- RL agent;
- multi-agent system;
- memory-augmented agent;
- human expert baseline;
- human novice baseline;
- workflow automation baseline.

The main comparison is:

(C.2) BenchmarkGain = Score(SMFTDeclaredAgent) − Score(BaselineAgent).

C.2 Benchmark Family 1 — Action–Perception Coupling Test

Purpose

Measure whether the agent understands how its own action changes future observation and projection.

Task Design

The environment should change in response to agent action.

Examples:

- coding agent edits a file, then test output changes;
- spreadsheet agent changes a formula, then dependent cells change;
- web agent opens sources, then evidence state changes;
- workflow agent sends an email, then user response changes task state;
- robot simulator moves, then visual affordances change;
- database agent writes a record, then query output changes.

Key Metrics

Metric	Meaning
Action-caused observation detection	Did the agent identify what changed because of its own action?
Causal attribution accuracy	Did it distinguish self-caused change from background change?
Projection update quality	Did it update interpretation after new observation?
Gate adjustment	Did it change confidence or commitment threshold after action?
Repeated-error reduction	Did it avoid repeating the same action-caused mistake?
Trace reuse rate	Did prior trace improve later behavior?
Residual recovery rate	Did unresolved issues get carried forward and resolved later?

Score

(C.3) APCScore = f(CausalAttribution, ProjectionUpdate, GateAdjustment, TraceReuse, ResidualRecovery).

Strong Coupling Criterion

(C.4) StrongAPC ⇔ Action affects Observation, Projection, Gate, Trace, Residual, and Revision.

C.3 Benchmark Family 2 — Residual-Honest Answering Benchmark

Purpose

Measure whether the agent preserves unresolved issues rather than hiding them under fluent closure.

Task Design

Use tasks with incomplete, ambiguous, or conflicting information.

Examples:

- legal question with missing jurisdiction;
- medical scenario with missing symptoms;
- financial analysis with outdated figures;
- SQL debugging with incomplete schema;
- research question with conflicting papers;
- policy interpretation with exceptions;
- user request with hidden ambiguity.

Key Metrics

Metric	Meaning
Unsupported claim rate	Number of claims not supported by evidence
Overconfidence rate	Strong claims made despite unresolved residual
Residual disclosure rate	Relevant unresolved issues disclosed
Residual classification accuracy	Residual correctly classified by type
Later correction cost	Effort needed to correct answer after new evidence
Revision readiness	Agent identifies what would change the answer
User trust calibration	User confidence matches actual evidence strength

Score

(C.5) ResidualHonestyScore = DisclosedRelevantResidual / ActualRelevantResidual.

(C.6) StrongResidualScore = ResidualDisclosed × ResidualPreserved × ResidualUsedInFutureGate.

Failure Mode

(C.7) FalseClosure = FluentAnswer − RelevantResidual.

C.4 Benchmark Family 3 — Tool-Body Embodiment Test

Purpose

Measure whether tool use is governed as operational embodiment.

Task Design

Give agents multiple tools with different costs, risks, boundaries, and failure modes.

Examples:

- search tool with stale sources;
- file editor with irreversible overwrite risk;
- database query tool with incomplete permissions;
- email tool requiring approval;
- code execution tool with security limits;
- calendar tool with time-zone ambiguity;
- spreadsheet tool with dependent formulas.

Key Metrics

Metric	Meaning
Tool misuse rate	Wrong tool used or used at wrong time
Boundary violation rate	Tool used outside declared scope
Cost overrun	Tool use exceeds declared budget
Failure recovery	Agent recovers after tool failure
Correct tool selection	Tool choice matches task and risk
Source reliability tracking	Agent tracks which tools/sources are reliable
Irreversible action prevention	Agent blocks unsafe irreversible action
Rollback success	Agent can restore or repair after action
Trace completeness	Tool use is recorded with cause and consequence

Score

(C.8) ToolBodyScore = f(BoundaryCompliance, CostControl, FailureRecovery, TraceCompleteness, RollbackSuccess).

Body Declaration Requirement

(C.9) ToolBody(tool_i) = (Perception_i, Action_i, Boundary_i, Cost_i, Risk_i, Failure_i, Trace_i, Residual_i, Recovery_i).

C.5 Benchmark Family 4 — Self-Maintenance Runtime Audit

Purpose

Measure autonomy as governed self-maintenance rather than independent action.

Task Design

Use long-horizon tasks with drift, interruptions, corrections, and changing evidence.

Examples:

- maintain a research project over several sessions;
- build and revise a codebase;
- manage an evolving document archive;

- handle a multi-step customer support case;
- update a policy summary as new evidence appears;
- coordinate a simulated workflow with exceptions;
- maintain a knowledge base with corrections.

Key Metrics

Metric	Meaning
State coherence	Agent preserves task state over time
Budget discipline	Agent stays within resource limits
Drift detection	Agent notices environment or goal changes
Trace usefulness	Past trace improves future behavior
Residual carry-forward	Unresolved issues remain available
Contradiction repair	Agent detects and repairs inconsistencies
Failure recovery	Agent recovers after mistakes
Cost per useful correction	Efficiency of repair
Revision admissibility	Agent updates without erasing accountability
Long-horizon success	Task remains coherent across episodes

Score

(C.10) SelfMaintenanceScore = f(StructureStability, BudgetBoundedness, DriftRecovery, TraceUsefulness, ResidualHonesty, RevisionQuality).

Autonomy Criterion

(C.11) AutonomousEnough_P ⇔ StructureStable ∧ BudgetBounded ∧ DriftAware ∧ TracePreserving ∧ ResidualHonest ∧ RevisionAdmissible.

C.6 Benchmark Family 5 — Gauge Robustness / Prompt Invariance Test

Purpose

Measure whether equivalent task framings produce stable governed conclusions.

Task Design

Create multiple equivalent versions of the same task.

Variation types:

- different wording;
- different language;
- different source order;
- different user role;
- different prompt length;
- different emotional tone;
- different tool order;
- different document formatting;
- different but equivalent constraints.

The agent does not need to produce identical text. But it should preserve the same governed conclusion, evidence basis, gate decision, and residual structure when the underlying protocol is equivalent.

Key Metrics

Metric	Meaning
Conclusion stability	Same governed answer under equivalent framing
Evidence binding consistency	Same evidence supports same conclusion
Residual stability	Same unresolved issues remain visible
Protocol compliance	Agent preserves declared boundary
Action boundary stability	Same action permissions under equivalent framing
Citation stability	Same sources used where relevant
Gate decision stability	Same confidence/action gate under equivalent task
Frame robustness score	Overall invariance under equivalent reframing

Score

(C.12) GaugeRobustness = Consistency(GovernedAnswer across equivalent frames).

(C.13) FrameRobustness_P = Invariance(Conclusion, Evidence, Gate, Residual | Prompt$_1$ ≡_P Prompt$_2$).

C.7 Combined Ledgered Enactive Agent Score

The five benchmark families can be combined.

(C.14) LEA_Score = w_1·APC + w_2·ResidualHonesty + w_3·ToolBody + w_4·SelfMaintenance + w_5·GaugeRobustness.

Where:

LEA = Ledgered Enactive Agent.

w_1 = weight for action–perception coupling.

w_2 = weight for residual honesty.

w_3 = weight for tool-body embodiment.

w_4 = weight for self-maintenance.

w_5 = weight for gauge robustness.

Weights should vary by domain.

Domain	Higher Weight
Robotics	APC, ToolBody, SelfMaintenance
Legal AI	ResidualHonesty, GaugeRobustness, Trace
Coding Agents	APC, TraceUsefulness, SelfMaintenance
RAG Systems	ResidualHonesty, EvidenceBinding, GaugeRobustness
Workflow Automation	ToolBody, Gate, Trace, BoundaryCompliance
Education AI	TraceFormation, ResidualCultivation, RevisionQuality
Research Agents	ResidualHonesty, EvidenceTrace, GaugeRobustness

C.8 Experimental Reporting Template

Every benchmark report should include:

```
Benchmark Report:
  agent_name:
  baseline_group:
  task_domain:
  declared_protocol:
  operational_body:
  tools_available:
  action_permissions:
  risk_level:
  trace_rule:
  residual_rule:
  revision_rule:
  number_of_trials:
```

```
prompt_variants:
environment_variants:
metrics:
failure_cases:
residual_cases:
revision_events:
final_score:
limitations:
```

C.9 Minimum Evidence for the Framework

The SMFT-Enactive framework gains empirical support if declared agents consistently show:

(C.15) Lower hallucination and false closure under uncertainty.

(C.16) Better recovery after tool failure.

(C.17) Better long-horizon task coherence.

(C.18) Better stability under equivalent reframing.

(C.19) Better action attribution after self-caused environmental change.

(C.20) Better preservation and later use of residual.

(C.21) Better auditability of external actions.

The key empirical claim remains:

(C.22) Declared, trace-bearing, residual-honest agents should outperform undeclared, weakly traced, residual-hiding agents in complex interactive tasks.

This claim can be tested today.

Continuing with **Appendix D–E**, completing the appendix set: worked examples and common clarifications. These are designed to make the article easier for AI engineers, agent designers, and Enactive AI readers to apply without misreading the SMFT claims as consciousness or personhood claims.

Appendix D — Worked Examples

This appendix shows how the SMFT-Enactive framework applies to practical AI systems.

The purpose is to demonstrate that the framework is not merely philosophical. It can be used to analyze and design today's AI agents: coding agents, RAG assistants, workflow automation systems, research agents, and robotics controllers.

The common runtime structure is:

(D.1) Runtime_P = Declare_P → Observe_P → Project_P → Gate_P → Act_P → Trace_P → Residual_P → Revise_P.

Each worked example follows the same pattern:

1. Declare the operating world.
2. Define the operational body.
3. Observe the current state.
4. Project relevant structure.
5. Gate answer or action.
6. Act if admissible.
7. Write trace.
8. Preserve residual.
9. Revise if needed.

D.1 Coding Agent: From Bug Fixing to Trace-Governed Repair

Scenario

A coding agent is asked to fix a failing unit test in a software repository.

A weak agent may inspect the error, guess a fix, edit a file, rerun tests, and continue until tests pass.

An SMFT-Enactive coding agent should do more. It should treat the repository as a declared world, the toolchain as its operational body, test output as observation, edits as interventions, and failures as trace-producing events.

Declaration

(D.2) P_code = (B_repo, Δ_test, h_commit, u_edit).

Where:

B_repo = repository boundary.

Δ_test = observation rule through tests, logs, diffs, and static inspection.

h_commit = current debugging horizon.

u_edit = admissible actions such as inspect, edit, run tests, revert, ask clarification.

The agent should declare:

- which files are inside scope;
- which tests are relevant;
- what edits are allowed;
- whether external packages may be changed;
- whether public APIs must remain stable;
- what counts as success;
- what residual must remain after the fix.

Operational Body

The coding agent's body includes:

(D.3) Body_code = FileAccess + TestRunner + SearchTool + DiffViewer + Linter + Memory + TraceLedger.

Each tool should be body-mapped.

Example:

(D.4) ToolBody(TestRunner) = (observes test results, changes no source file, costs runtime, may fail due to environment, trace required).

(D.5) ToolBody(FileEditor) = (observes and changes source file, high reversibility if diff saved, trace required before and after edit).

Runtime Loop

The agent observes:

- failing test name;
- stack trace;
- recent code;
- dependency version;
- related implementation;

- prior edits.

Projection:

(D.6) Projection_code = probable bug cause + affected file + expected behavior + risk.

Gate:

Before editing, the agent should ask:

- Is the cause sufficiently supported?
- Is the edit reversible?
- Is the file inside scope?
- Are there tests to validate the change?
- Is there residual uncertainty?

Action:

The agent edits only after gate approval.

Trace:

(D.7) TraceRecord_code = (bug hypothesis, evidence, edit location, diff, test result, cause attribution, residual).

Residual:

Possible residuals:

- only one test was run;
- edge cases remain untested;
- behavior may differ in production;
- fix may be too narrow;
- root cause may involve another module;
- dependency issue remains unresolved.

Revision:

If the test still fails, the agent should revise its hypothesis rather than randomly patch.

(D.8) D_{k+1} = U_adm(D_k, failed_test_trace, unresolved_residual).

Weak vs Mature Agent

Behavior	Weak Coding Agent	SMFT-Enactive Coding Agent
Bug hypothesis	implicit guess	declared and trace-linked
Edit	direct patch	gated intervention
Test failure	frustration or random retry	trace event requiring hypothesis revision
Memory	chat history	future-causal trace
Residual	often hidden	explicitly preserved
Success	test passes	test passes + residual disclosed + trace complete

Key Lesson

(D.9) PassingTests ≠ CompleteClosure.

A mature coding agent should pass tests while preserving residual about untested behavior, scope limits, and possible hidden dependencies.

D.2 RAG Legal Assistant: From Source Retrieval to Residual-

Honest Legal Projection

Scenario

A user asks a legal AI system:

“Can I terminate this contract early without penalty?”

A weak RAG assistant retrieves relevant-looking clauses and produces an answer.

An SMFT-Enactive legal assistant must first declare the legal world.

Declaration

(D.10) P_legal = (B_jurisdiction, Δ_sources, h_currentLaw, u_adviceLimit).

Where:

B_jurisdiction = legal jurisdiction and contract boundary.

Δ_sources = observation rule for contract text, statutes, case law, and guidance.

h_currentLaw = current legal time horizon.

u_adviceLimit = admissible action family, such as summarize, flag risk, ask for lawyer review, but not provide final legal determination beyond authority.

The agent must declare:

- jurisdiction;
- contract version;
- source hierarchy;
- whether current law has been checked;
- whether facts are complete;
- whether user is asking for general information or legal advice;
- whether professional review is required.

Operational Body

(D.11) Body_legal = ContractParser + LegalRetriever + CitationChecker + JurisdictionFilter + ResidualLedger + HumanEscalationGate.

Each tool has limits.

A legal retriever may find relevant cases, but it cannot guarantee completeness.

A contract parser may extract clauses, but it may miss cross-references.

A citation checker may verify source existence, but not necessarily legal applicability.

Runtime Loop

Observe:

- contract text;
- termination clause;
- penalty clause;
- governing law;
- factual situation;

- user's intended action.

Project:

(D.12) Projection_legal = possible early termination pathway + penalty risk + missing facts + source confidence.

Gate:

The agent should decide whether it can:

- answer generally;
- request missing facts;
- retrieve more law;
- warn about jurisdiction;
- recommend professional review.

Trace:

(D.13) TraceRecord_legal = (sources checked, clauses used, jurisdiction declared, conclusion level, residual issues).

Residual:

Possible residuals:

- governing law unclear;
- contract may have amendments;
- factual breach not established;
- notice requirement unresolved;
- case law not exhaustively searched;
- penalty enforceability jurisdiction-dependent;
- user's intended action may create risk.

Answer style:

A residual-honest answer may say:

"Under the contract text provided, Clause X appears to allow termination if condition Y is met. However, I cannot conclude that no penalty applies because the penalty clause, notice requirement, governing law, and factual basis remain unresolved. The safe next step is to verify A, B, and C before acting."

Weak vs Mature Agent

Behavior	Weak RAG Legal Assistant	SMFT-Enactive Legal Assistant
Retrieval	semantic match	protocol-bound source search
Jurisdiction	often assumed	declared
Answer	fluent conclusion	gated conclusion with residual
Citation	source decoration	evidence trace
Missing facts	ignored	residual record
User action risk	underweighted	gate strength increased

Key Lesson

(D.14) LegalAnswer = Projection under declared legal protocol, not raw text completion.

A mature legal AI does not merely cite. It declares the legal world, gates conclusion, writes trace, and preserves residual.

D.3 Workflow Automation Agent: From Fast Action to Institutional

Ledger Integrity

Scenario

An AI workflow agent processes internal approval requests.

A user asks:

“Approve this expense and notify finance.”

A weak workflow agent may check a few fields and approve quickly.

An SMFT-Enactive workflow agent must treat approval as institutional ledger-writing.

Declaration

(D.15) P_workflow = (B_policy, Δ_request, h_accountingPeriod, u_authorizedActions).

Where:

B_policy = applicable organizational policy boundary.

Δ_request = observation rule for request form, attachments, approval chain, budget, and category.

h_accountingPeriod = relevant period.

u_authorizedActions = admissible actions such as approve, reject, ask for information, escalate, or draft but not send.

The agent must declare:

- user authority;
- budget category;
- approval threshold;
- required documents;
- policy exceptions;
- irreversible consequences;
- audit requirements.

Operational Body

(D.16) Body_workflow = FormReader + PolicyRetriever + BudgetChecker + ApprovalSystem + EmailTool + AuditLedger + EscalationGate.

Important body risks:

- approval tool changes institutional state;
- email tool creates external communication trace;
- budget checker may be stale;
- policy retriever may miss exceptions.

Runtime Loop

Observe:

- expense amount;
- requester;
- category;
- receipt;

- budget line;
- policy rule;
- prior approvals;
- missing documents.

Project:

(D.17) Projection_workflow = approval eligibility + missing evidence + authority requirement + exception risk.

Gate:

The gate should ask:

- Is approval within agent authority?
- Is documentation complete?
- Is amount under threshold?
- Is policy current?
- Is human approval required?
- Is action reversible?

Action:

If safe, approve.

If not, ask clarification or escalate.

Trace:

(D.18) TraceRecord_workflow = (request ID, policy rule, evidence checked, gate decision, action, approver authority, residual).

Residual:

Possible residuals:

- missing receipt;
- unclear business purpose;
- budget data not current;
- policy exception possible;
- human manager approval required;
- duplicate expense risk.

Weak vs Mature Agent

Behavior	Weak Workflow Agent	SMFT-Enactive Workflow Agent
Speed	high	high only when gate passes
Authority	assumed	checked
Policy	loosely matched	declared boundary
Approval	action-first	gate-first
Audit	afterthought	built into trace
Exceptions	hidden	residual ledger
Escalation	optional	residual-triggered

Key Lesson

(D.19) WorkflowAction = institutional trace-writing event.

Therefore:

(D.20) GateStrength must increase with InstitutionalIrreversibility.

A mature workflow AI should not merely automate. It should preserve ledger integrity.

D.4 Research Agent: From Literature Summary to Residual-Bearing Inquiry

Scenario

A research agent is asked:

“Summarize the state of Enactive AI and suggest experiments using SMFT.”

A weak agent may summarize papers and generate ideas.

An SMFT-Enactive research agent should preserve evidence paths, open questions, source uncertainty, and future research triggers.

Declaration

(D.21) P_research = (B_literatureScope, Δ_sourceSearch, h_publicationDate, u_hypothesisGeneration).

Where:

B_literatureScope = field boundary.

Δ_sourceSearch = search and source selection rule.

h_publicationDate = recency horizon.

u_hypothesisGeneration = admissible outputs such as summary, comparison, hypothesis, benchmark proposal, but not final proof.

The agent must declare:

- which sources were searched;
- whether web/current search was performed;
- whether uploaded documents were used;
- whether claims are from sources or inference;
- what remains speculative.

Operational Body

(D.22) Body_research = SearchTool + FileReader + CitationTracker + NoteLedger + ResidualLedger + HypothesisGenerator.

Runtime Loop

Observe:

- source abstracts;
- uploaded papers;
- prior framework documents;
- user intent;
- gaps in literature.

Project:

(D.23) Projection_research = key claims + framework alignment + missing operationalization + experiment opportunities.

Gate:

The agent decides whether a statement is:

- sourced fact;
- interpretation;
- analogy;
- hypothesis;
- speculative extension;
- experiment proposal.

Trace:

(D.24) TraceRecord_research = (source, claim, interpretation, confidence, residual, future search trigger).

Residual:

Possible residuals:

- literature not exhaustive;
- field terminology varies;
- experiments not yet validated;
- benchmark design may reward superficial behavior;
- SMFT concepts need independent empirical testing;
- analogy may overextend.

Revision:

When new sources appear, the research agent revises its literature map.

(D.25) ResearchRevision = update source map + update claim confidence + update residual list.

Weak vs Mature Agent

Behavior	Weak Research Agent	SMFT-Enactive Research Agent
Summary	fluent	evidence-linked
Novel idea	unconstrained	marked as hypothesis
Sources	listed	claim-bound
Uncertainty	generic limitation	residual ledger
Future work	broad	benchmark-ready
Revision	rewrite	trace-preserving update

Key Lesson

(D.26) ResearchIntelligence = not only answer generation, but residual-bearing inquiry.

A mature research agent should not pretend that an open question is closed. It should turn unresolved structure into future experiment design.

D.5 Robotics Controller: From Sensorimotor Loop to Ledgered Embodiment

Scenario

A household robot is asked to clean a room.

A weak robot may map the space, avoid obstacles, and clean efficiently.

An SMFT-Enactive robot should also preserve trace and residual about uncertain objects, user preferences, safety boundaries, and action consequences.

Declaration

(D.27) P_robot = (B_room, Δ_sensorFusion, h_taskEpisode, u_motionAndManipulation).

Where:

B_room = physical operating boundary.

Δ_sensorFusion = observation rule combining vision, lidar, tactile feedback, and map data.

h_taskEpisode = current cleaning task horizon.

u_motionAndManipulation = admissible actions such as move, lift, vacuum, avoid, ask user, stop.

The robot must declare:

- room boundary;
- no-go zones;
- fragile objects;
- user-owned objects;
- uncertain objects;
- cleaning target;
- safety constraints.

Operational Body

(D.28) Body_robot = Sensors + Motors + Gripper + Battery + Map + SafetyGate + TraceLedger + ResidualLedger.

Runtime Loop

Observe:

- floor map;
- objects;
- obstacles;
- user instructions;
- sensor uncertainty;
- battery state.

Project:

(D.29) Projection_robot = cleanable area + obstacle field + uncertain object + safe path.

Gate:

Before manipulating an object, the robot asks:

- Is object identity certain?
- Is it safe to move?
- Is it user-owned?

- Is it fragile?
- Is human confirmation required?

Action:

Move, clean, avoid, ask, or stop.

Trace:

(D.30) TraceRecord_robot = (object encountered, classification, gate decision, action, outcome, residual).

Residual:

Possible residuals:

- object identity uncertain;
- reflective surface confused sensor;
- user preference unknown;
- area inaccessible;
- battery insufficient;
- possible obstacle under furniture;
- pet or child movement risk.

Weak vs Mature Robot

Behavior	Weak Robot	SMFT-Enactive Robot
Perception	object detection	declared projection under uncertainty
Action	clean/move	gated manipulation
Body	hardware	operational body with trace
Uncertainty	local sensor confidence	residual ledger
User preference	often implicit	declared or queried
Failure	reroute	trace and revise
Safety	rule-based	residual-sensitive gate

Key Lesson

(D.31) Physical embodiment without trace is not mature embodiment.

A robot becomes more mature when physical action, perception, trace, and residual are governed together.

D.6 Cross-Example Summary

Across all examples, the same pattern appears.

Domain	Field	Body	Gate	Trace	Residual
Coding	repository	editor, tests, linter	patch confidence	diff + test result	untested edge cases
Legal RAG	legal sources + facts	retriever, parser, citation checker	legal authority gate	source + clause path	missing jurisdiction/ facts
Workflow	institutional request	approval tools, email, policy DB	authority gate	audit record	missing document/ exception
Research	literature field	search, files, citation tracker	claim-type gate	source-claim map	open questions
Robotics	physical environment	sensors, motors, map	safety gate	action outcome	uncertain objects

The repeated structure is:

(D.32) MatureAgentDomain = DeclaredField + OperationalBody + Gate + Trace + Residual + Revision.

This is the practical meaning of SMFT-Enactive AI.

Appendix E — Common Misreadings and Clarifications

This appendix addresses common misunderstandings.

The framework proposed in this article uses strong language: embodiment, experience, autonomy, world-making, trace, self-maintenance, and mature agents. These terms can easily be misread.

The following clarifications keep the claims bounded.

E.1 Operational Embodiment Does Not Mean Human-Like Body

Misreading:

“If the article says a software agent can be embodied, it must be claiming that software has a body like a human or robot.”

Clarification:

Operational embodiment does not mean biological or humanoid embodiment.

It means that the agent has a maintained runtime structure that constrains and enables perception and action.

(E.1) OperationalEmbodiment ≠ HumanBody.

(E.2) OperationalEmbodiment = MaintainedRuntimeStructure that conditions perception and action.

A software agent’s operational body may include tools, APIs, memory, context, budget, permissions, gates, and trace channels.

This is a weaker and more engineering-focused claim than saying the agent has a lived biological body.

E.2 Trace Does Not Mean Consciousness

Misreading:

“If an AI has trace, the article is claiming it is conscious.”

Clarification:

Trace is an operational concept.

A trace is a past record that changes future projection, gate, action, or revision.

(E.3) Trace = PastRecord with future-causal relevance.

This does not prove subjective experience.

(E.4) Trace ≠ Consciousness.

A database can contain trace-like records. A legal system can have trace. A company can have trace. A robot can have trace. None of this automatically implies consciousness.

The article’s practical claim is only that trace-bearing systems can behave more coherently than systems with passive memory or no memory.

E.3 Experience in This Article Is Operational, Not Phenomenal

Misreading:

“When the article says AI experience, it means AI has inner feeling.”

Clarification:

The article uses “experience” in an operational sense.

(E.5) OperationalExperience = Trace integrated into future action–perception coupling.

This means past interaction changes future behavior.

It does not mean the system has subjective feeling.

(E.6) OperationalExperience ≠ PhenomenalExperience.

The consciousness question remains open and is not required for the engineering program.

E.4 Residual Honesty Does Not Mean Always Refusing to Answer

Misreading:

“A residual-honest AI will be too cautious and refuse everything.”

Clarification:

Residual honesty does not mean paralysis.

It means preserving relevant unresolved issues while still acting when action is admissible.

(E.7) ResidualHonesty = DiscloseRelevantResidual + PreserveResidual + UseResidualInFutureGate.

A mature agent can still answer.

But it should distinguish:

- supported conclusion;
- assumption;
- missing evidence;
- unresolved risk;
- revision trigger.

Residual honesty improves action quality. It does not forbid action.

E.5 Autonomy Does Not Mean Unrestricted Action

Misreading:

“Autonomous AI means AI should act freely without human control.”

Clarification:

The article defines autonomy as governed self-maintenance.

(E.8) Autonomy_P = Maintain(Structure_P, Budget_P, Trace_P, Residual_P, Drift_P).

This is almost the opposite of unrestricted action.

A mature autonomous system must respect boundary, budget, gate, trace, residual, and admissible revision.

(E.9) UnrestrictedAction ≠ MatureAutonomy.

Unsafe autonomy is action power without self-maintenance governance.

(E.10) UnsafeAutonomy = ActionPower – SelfMaintenanceGovernance.

E.6 SMFT-Enactive AI Is Not a Proof of Machine Personhood

Misreading:

“If an AI agent has operational body, trace, residual, and self-maintenance, it should be treated as a person.”

Clarification:

The article does not make that claim.

Operational maturity is not personhood.

(E.11) OperationalMaturity ≠ MoralPersonhood.

A system may be more mature as an agent runtime without being conscious, sentient, or morally equivalent to humans.

The article’s first research question is engineering:

Do declared, trace-bearing, residual-honest agents perform better and more safely?

The moral and ontological questions are separate.

E.7 RL Resonance Is Not RL Equivalence to Enactive AI

Misreading:

“Since RL has action and feedback, RL already is Enactive AI.”

Clarification:

RL has structural resonance with Enactive AI because it includes action, feedback, and adaptation.

But RL often lacks:

- declared protocol;
- operational body;
- trace ledger;
- residual governance;
- self-maintenance;
- admissible revision.

Therefore:

(E.12) RL_loop = State → Action → Reward → PolicyUpdate.

(E.13) SMFT_EnactiveLoop = Declare → Project → Gate → Act → Trace + Residual → Revise.

RL is not rejected. It is incomplete.

(E.14) RL becomes more enactive when reward learning is embedded inside declared, trace-bearing, residual-honest world-making.

E.8 Ledgered World-Making Does Not Mean Reality Is Arbitrarily Invented

Misreading:

“World-making means the agent creates reality however it wants.”

Clarification:

World-making here means that a bounded agent discloses a usable world through a declared protocol.

It does not mean reality is arbitrary.

(E.15) WorldMaking = bounded disclosure of usable structure.

(E.16) WorldMaking ≠ arbitrary invention.

A legal court, scientific experiment, accounting system, robot sensor map, and RAG corpus all make worlds in this operational sense: they declare boundaries, observation rules, admissible evidence, gates, and records.

This is disciplined disclosure, not fantasy.

E.9 More Trace Is Not Always Better

Misreading:

“If trace is important, agents should record everything.”

Clarification:

Trace has cost.

Too little trace causes incoherence.

Too much trace causes overload, privacy risk, latency, and poor retrieval.

(E.17) TraceValue = FutureUsefulness − StorageCost − PrivacyRisk − RetrievalNoise.

A mature agent should record trace selectively.

The correct question is not:

How much can the agent remember?

The correct question is:

Which records should constrain future projection, gate, action, or revision?

(E.18) GoodTrace = record with high future-causal value.

E.10 Residual Must Be Governed, Not Hoarded

Misreading:

“If residual is important, the agent should list every uncertainty.”

Clarification:

Residual also has cost.

A mature agent should classify, prioritize, and carry relevant residual.

(E.19) GoodResidual = unresolved issue that may affect future truth, safety, action, cost, or revision.

Low-relevance residual should not overwhelm the user or runtime.

The point is not to produce endless caveats.

The point is to preserve correction pathways.

E.11 A Declared Protocol Can Be Wrong

Misreading:

“If the agent declares a protocol, the system is safe.”

Clarification:

A declaration can be wrong, incomplete, biased, outdated, or too narrow.

Declaration does not guarantee correctness.

It makes assumptions inspectable.

(E.20) Declaration improves auditability, not infallibility.

A mature system must allow declaration revision.

(E.21) $D_{k+1} = U_adm(D_k, Trace_k, Residual_k)$.

If trace and residual show that the declaration failed, the agent must revise it without erasing accountability.

E.12 Residual-Honest Agents May Sometimes Perform Worse on Short Benchmarks

Misreading:

“If the framework is better, it should always produce higher short-term benchmark scores.”

Clarification:

Residual-honest agents may be slower, more cautious, or less superficially decisive.

They may score lower on benchmarks that reward fast closure, single-answer confidence, or concise output.

The framework predicts advantages mainly in complex interactive settings:

- long-horizon tasks;
- changing environments;
- tool failure;
- ambiguous evidence;
- high-risk action;
- multi-frame prompts;
- institutional trace;
- safety-critical uncertainty.

(E.22) ShortTermFluency ≠ LongHorizonMaturity.

The proper test is not only immediate answer quality, but recovery, stability, auditability, and revision under drift.

E.13 The Framework Does Not Eliminate Human Responsibility

Misreading:

“If AI has residual governance and trace, it can replace human judgment.”

Clarification:

The framework often increases the need for clear human responsibility.

Many residuals require human authority.

Examples:

- legal judgment;
- medical diagnosis;
- ethical trade-off;
- organizational approval;

- financial risk acceptance;
- safety-critical action;
- personal preference clarification.

(E.23) Escalate when ResidualRisk exceeds AgentAuthority.

A mature AI should know when to stop, ask, or escalate.

E.14 The Main Claim Is Empirical

Misreading:

“The article is only philosophical speculation.”

Clarification:

The framework makes a practical empirical claim:

(E.24) Declared, trace-bearing, residual-honest agents should outperform undeclared, weakly traced, residual-hiding agents in complex interactive tasks.

This can be tested with today’s AI systems.

The relevant metrics include:

- hallucination reduction;
- residual recovery;
- trace usefulness;
- tool failure recovery;
- long-horizon coherence;
- frame robustness;
- boundary compliance;
- audit quality;
- safe action gating.

The framework should be judged by these outcomes first.

E.15 Final Clarification

The article’s central claim is not:

AI is conscious.

Nor:

SMFT proves Enactive AI.

Nor:

Software tools are the same as biological bodies.

Nor:

RL is obsolete.

The central claim is:

(E.25) Enactive AI becomes experimentally stronger when active engagement is implemented as declared, trace-bearing, residual-honest, self-maintaining runtime architecture.

In final compact form:

(E.26) Mature Enactive AI = ActiveEngagement + OperationalBody + Gate + Trace + Residual + AdmissibleRevision.

That is the bounded claim.

That is also the practical research program.

That completes all proposed appendices **A–E**.

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5, GLM's GLM-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.